Advanced and Distributed Algorithms
Geometric algorithms

Dr. Benoît PIRANDA

Ph.D. Associate Professor HDR

Member of Computer Science Department of Femto-ST institute

benoit.piranda@femto-st.fr

Outline

- 2D Algorithmic
  - Convexity and non-convexity
  - Convex hull
  - Delaunay triangulation
  - Creating a regular mesh for a set of points
  - Voronoï diagram
- 3D Algorithmic
  - Geometric modeling using CSG trees
  - Object picking / ray-tracing
  - Z-Buffer algorithm, OpenGL library

Basic 2D functions

- Distance between two points
  - Distance from A to B

$$d(A,B) = \left\|\overrightarrow{AB}\right\| = \sqrt{(B_x - A_x)^2 + \left(B_y - A_y\right)^2}$$

- Orthogonal direction in 2D
  - $\vec{v}$ Orthogonal direction (to the right) of $\vec{u}$

$$\vec{v}=\vec{u} \wedge \vec{z} = \begin{pmatrix} u_x \\ u_y \\ 0 \end{pmatrix} \wedge \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} u_y \\ -u_x \\ 0 \end{pmatrix}$$

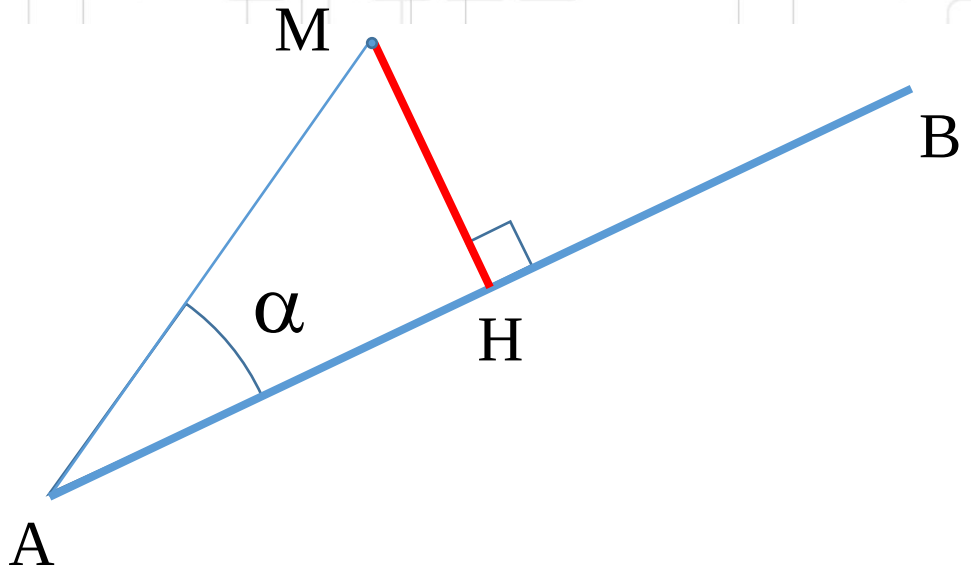
Basic 2D functions

- Distance from a point M to a line (AB):

$$\overrightarrow{AB} \times \overrightarrow{AM} = \sin\alpha \times \left\|\overrightarrow{AB}\right\| \times \left\|\overrightarrow{AM}\right\| \times \vec{z}$$

$$\left\|\overrightarrow{MH}\right\| = \sin\alpha \times \left\|\overrightarrow{AM}\right\| = \frac{\left\|\overrightarrow{AB} \times \overrightarrow{AM}\right\|}{\left\|\overrightarrow{AB}\right\|}$$

$$\overrightarrow{AB} \times \overrightarrow{AM} = \begin{pmatrix} 0 \\ 0 \\ AB_x AM_y - AB_y AM_x \end{pmatrix}$$



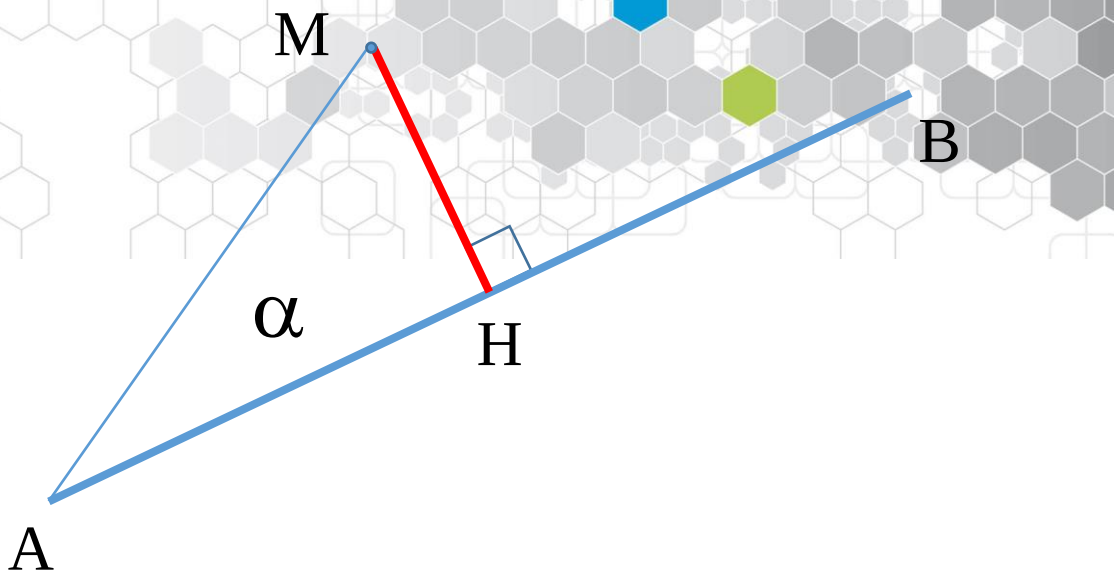
$$\left\|\overrightarrow{MH}\right\| = \frac{AB_x AM_y - AB_y AM_x}{\sqrt{AB_x^2 + AB_y^2}}$$

Basic 2D functions

- Distance to a segment: $H \in [AB]$
  - Compute the position of H in [AB]

$$\overrightarrow{AB} \cdot \overrightarrow{AM} = \cos\alpha \times \left\|\overrightarrow{AB}\right\| \times \left\|\overrightarrow{AM}\right\|$$

$$\left\|\overrightarrow{AH}\right\| = \cos\alpha \times \left\|\overrightarrow{AM}\right\| = \frac{\left\|\overrightarrow{AB} \cdot \overrightarrow{AM}\right\|}{\left\|\overrightarrow{AB}\right\|}$$



$$\left\|\overrightarrow{AH}\right\| = \frac{AB_x AM_x + AB_y AM_y}{\sqrt{AB_x^2 + AB_y^2}}$$

```
QPair<double,Vector2D> distanceToEdge(const Vector2D &M, int i) {
    Vector2D AB = tabPts[i+1]-tabPts[i];
    Vector2D AM = M-tabPts[i];
    double lAB = AB.length();
    double AH = (AB.x*AM.x + AB.y*AM.y)/lAB;
    if (AH<0) return QPair<double,Vector2D>(AM.length(),tabPts[i]);
    else if (AH>lAB) return QPair<double,Vector2D>((M-tabPts[i+1]).length(),tabPts[i+1]);
    else {
        Vector2D MH=tabPts[i]+AB*(AH/lAB)-M;
        return QPair<double,Vector2D>(MH.length(),M+MH);
    }
}
```

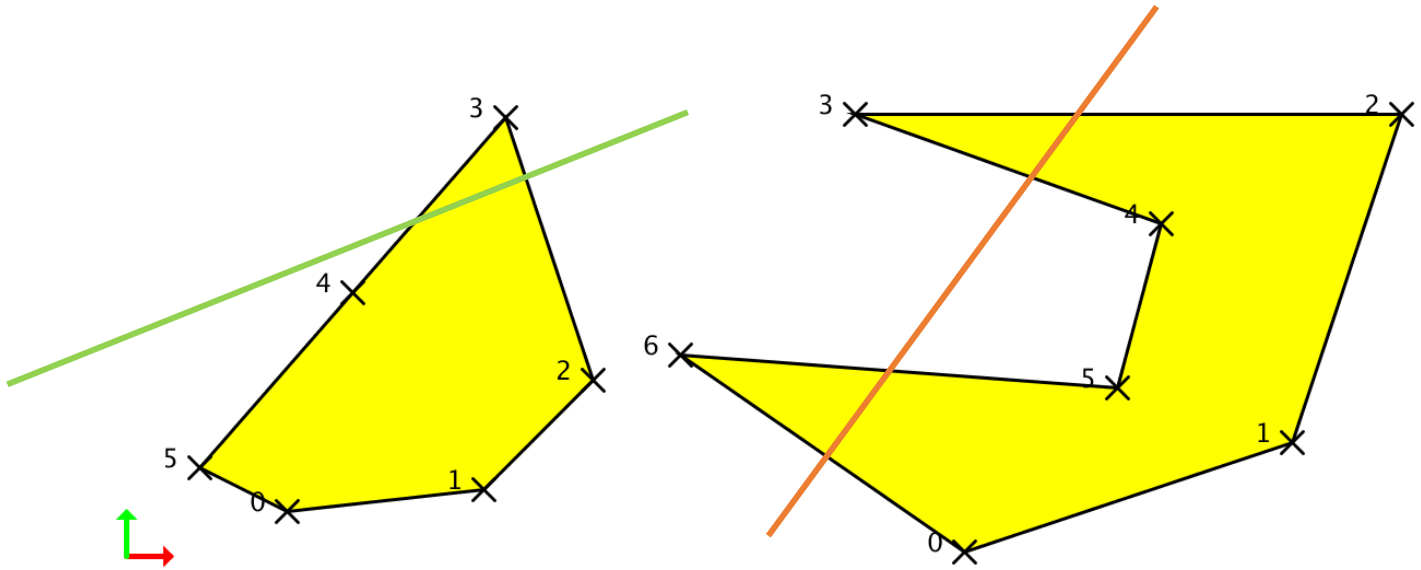
Geometric algorithms on polygons

Definitions

- Definitions
  - **A polygon is a plane figure** that is described by a finite number of straight line segments connected to form a closed polygonal chain or polygonal circuit.
  - The segments of a polygonal circuit are called its **edges** or **sides**, and the points where two edges meet are the polygon's **vertices** or corners.
  - A **simple polygon** is one which does not intersect itself.

Convexity and non-convexity

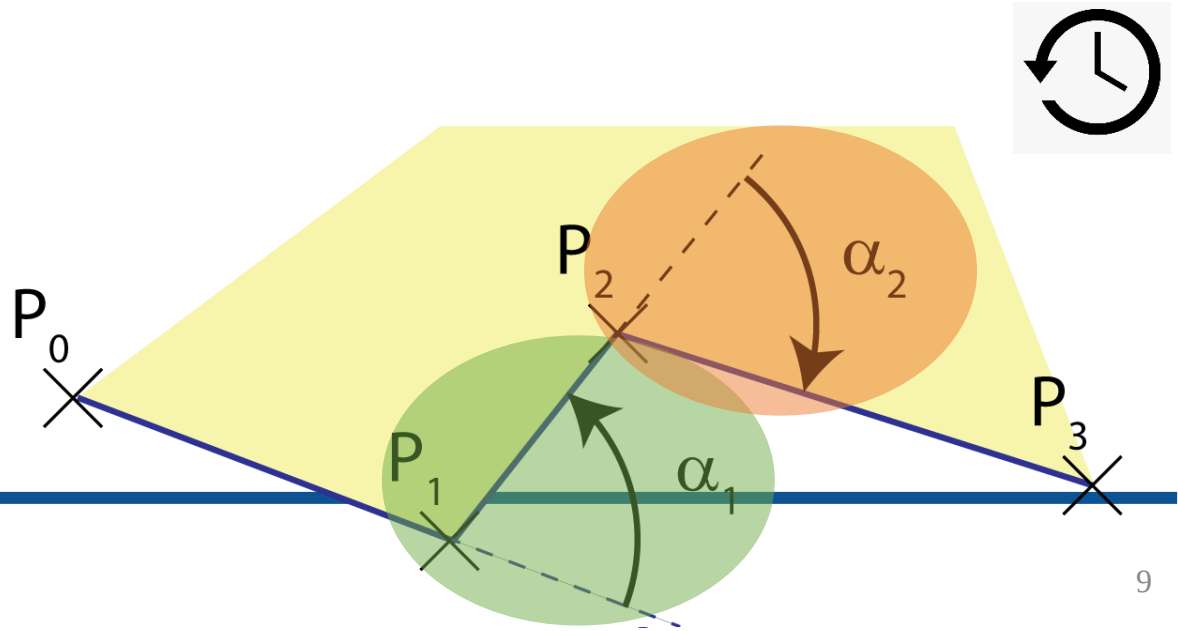
- Propriety #1
  - A simple polygon is **convex** if **any line** drawn through the polygon (and not tangent to an edge or corner) meets its boundary **exactly twice**.



Convexity and non-convexity

- Propriety #2
  - A polygon is concave if there is at least one interior angle greater than 180°.
  - It exists $\alpha_i = \overrightarrow{P_{i-1}P_i}\ \overrightarrow{P_iP_{i+1}} < 0$
- Convexity algorithm:
  - We define a polygon by a list of N 2D vertices $P_i \mid i \in [0..N-1]$ ordered counterclockwise.

$$sign(\alpha_i) = sign\left(\left[\overrightarrow{P_{i-1}P_i} \wedge \overrightarrow{P_iP_{i+1}}\right]_z\right)$$



Convexity algorithm

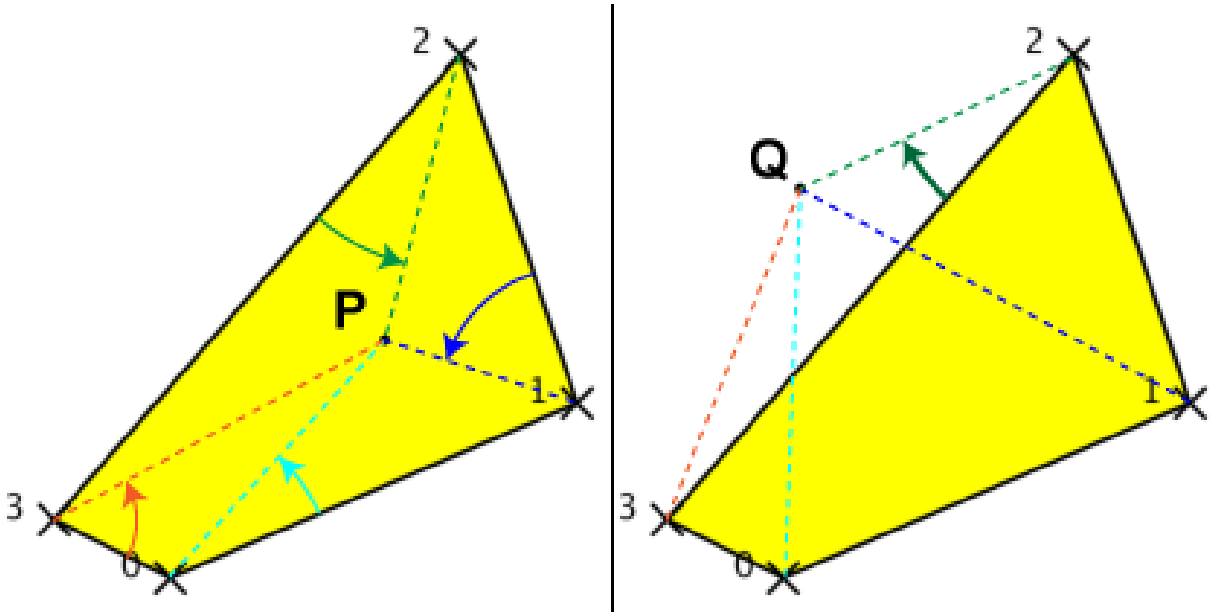
- The polygon is stored in a array **tabPts** of **N+1** vertices (with P[N]=P[0]).

```
bool Polygon::isOnTheLeft(const Vector2D &P,int i) {
  Vector2D AB = tabPts[i+1]-tabPts[i],
           AP = P-tabPts[i];
  return (AB.x*AP.y - AB.y*AP.x)>=0;
}

bool Polygon::isConvex() {
  int i=0;
  while (i<N && isOnTheLeft(tabPts[(i+2)%N],i)) {
    i++;
  }
  return (i==N);
}
```

- Complexity of this algorithm? **O(N)**

Position of point / a convex polygon

- Due to counterclockwise order of the vertices, a point *P* is in the **interior of a convex polygon** if *P* is on the left of every edges:

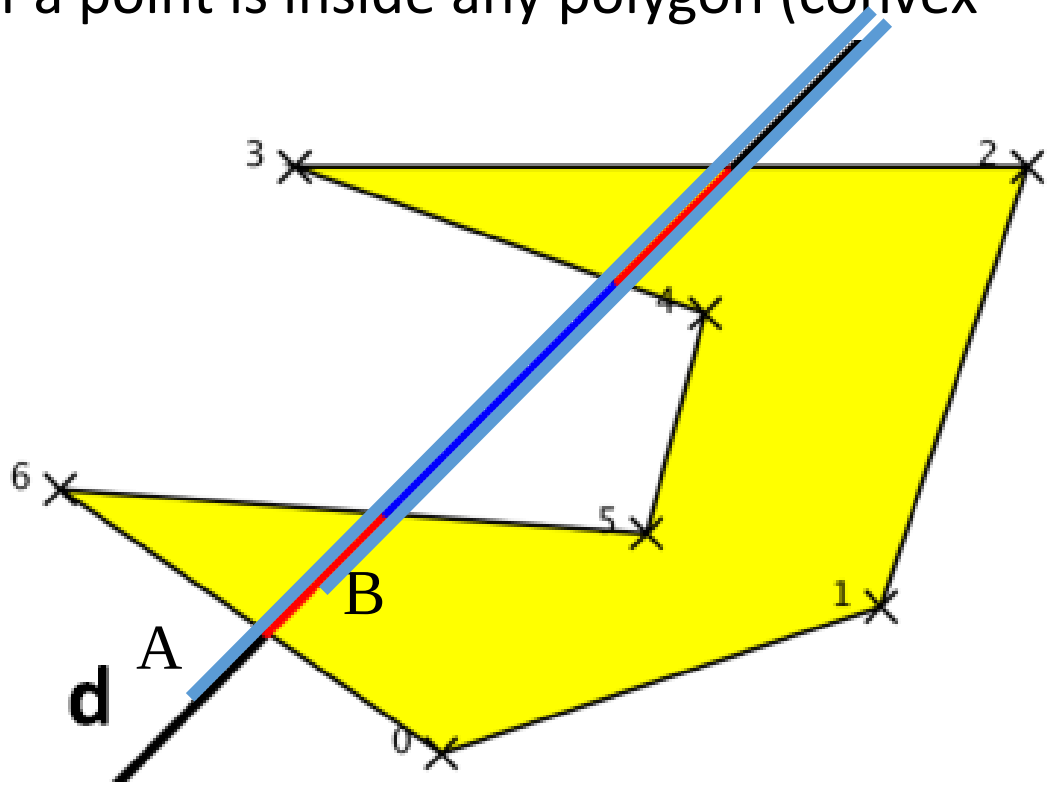


$$P_i P_{i+1} \mid i \in [0..N]$$

```
bool Polygon::isInside(const Vector2D
&P) {
int i=0;
  while (i<N && isOnTheLeft(P,i)) {
    i++;
  }
  return (i==N);
}
```

Position of point / a concave polygon

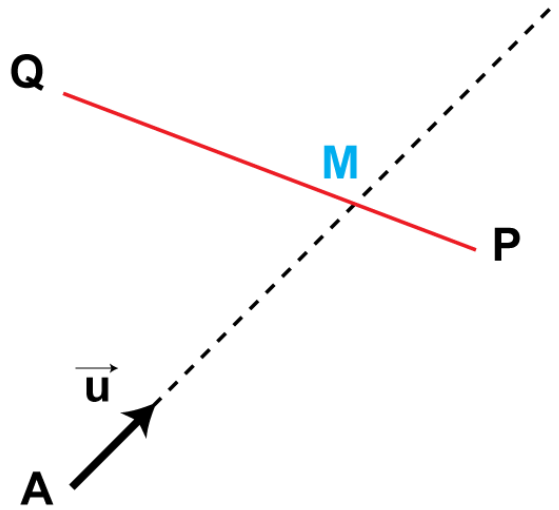
- Calculation of the intersection points between a line and a polygon
  - How many points between a line and a polygon may we get?
  - Demonstrate that this number is even!
  - Deduce an algorithm allowing to check if a point is inside any polygon (convex or concave)!
  - Complexity? **O(N)**



Intersection between a ray and a segment

- Let $[A\vec{u})$ be a ray (half-line), $[PQ]$ a segment, we search the point $\mathrm{M} = [A\vec{u}) \cap [PQ]$

$$\begin{cases} \overrightarrow{AM} = k_1\vec{u} \\ k_1 \geq 0 \\ \overrightarrow{PM} = k_2\overrightarrow{PQ} \\ 0 \leq k_2 \leq 1 \end{cases} \rightarrow \begin{cases} \overrightarrow{PA} + k_1\vec{u} = k_2\overrightarrow{PQ} \\ k_1 \geq 0 \\ 0 \leq k_2 \leq 1 \end{cases}$$



We deduce from this system, 2 equations and 2 unknowns:

$$\begin{pmatrix} u_x & -PQ_x \\ u_y & -PQ_y \end{pmatrix}\begin{pmatrix} k_1 \\ k_2 \end{pmatrix} = \begin{pmatrix} AP_x \\ AP_y \end{pmatrix} \rightarrow \begin{pmatrix} k_1 \\ k_2 \end{pmatrix} = \begin{pmatrix} u_x & -PQ_x \\ u_y & -PQ_y \end{pmatrix}^{-1}\begin{pmatrix} AP_x \\ AP_y \end{pmatrix}$$

Intersection between a ray and a segment

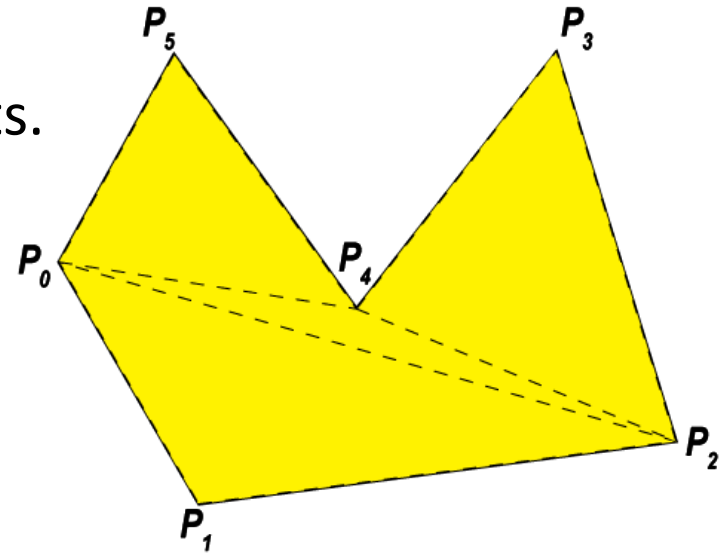
- We get:

$$\begin{cases} k_1 = \dfrac{AP_xPQ_y - AP_yPQ_x}{u_xPQ_y - u_yPQ_x} \\ k_1 \geq 0 \\ k_2 = \dfrac{AP_xu_y - AP_yu_x}{u_xPQ_y - u_yPQ_x} \\ 0 \leq k_2 \leq 1 \end{cases}$$

- A is in the border if:
  - $\exists i \mid (\mathrm{A} = P_i)$
  - $\left(AP_xPQ_y - AP_yPQ_x = 0\right) \wedge \left(0 \leq AP_xu_y - AP_yu_x \leq u_xPQ_y - u_yPQ_x\right)$
- Non collinearity condition $u_xPQ_y - u_yPQ_x \neq 0$
  - Warning: if $[A\vec{u})$ is collinear to an edge $[P_iP_{i+1}]$, there is no intersection with this edge.

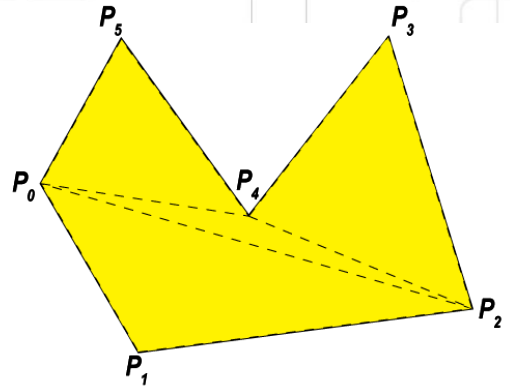
Simple triangulation algorithms

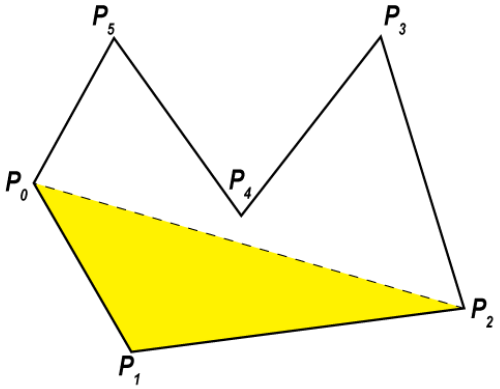
- Goal: transform a concave polygon into a set of convex polygons
- Ear clipping method:
  1. Searching a triangle $(P_i P_{i+1} P_{i+2})$ which does not contains any other point of the polygon.
  2. Remove the point P_{i+1} from the polygon.
  3. Steps 1 and 2 are repeated until it remains only 2 points.

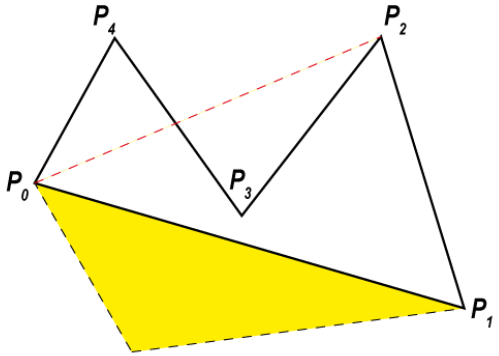


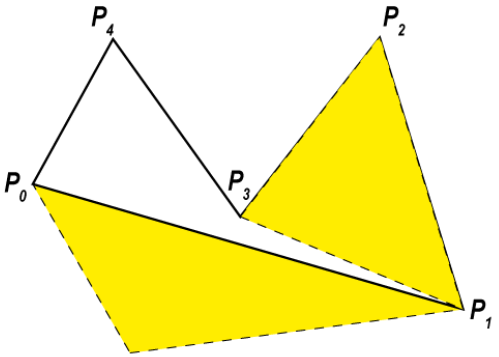
Ear clipping algorithm

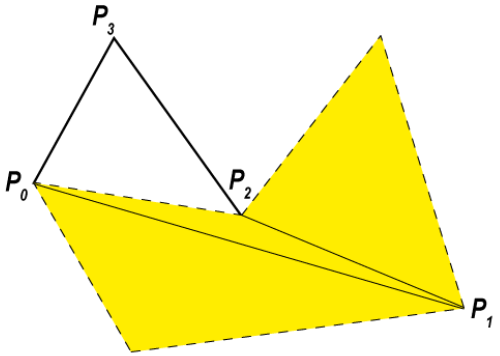
1. Searching a triangle $(P_i P_{i+1} P_{i+2})$ which does not contains any other point of the polygon, $(i \in [0..n-2])$.
2. Remove the point P_{i+1} from the polygon (indices).
3. Steps 1 and 2 are repeated until it remains only 2 points.

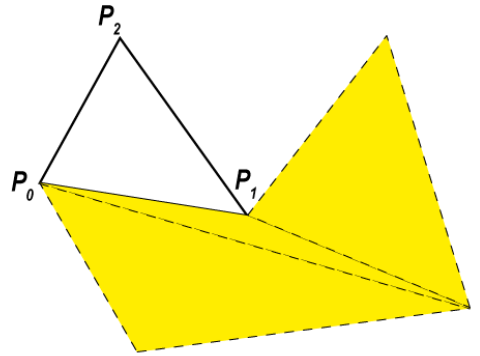












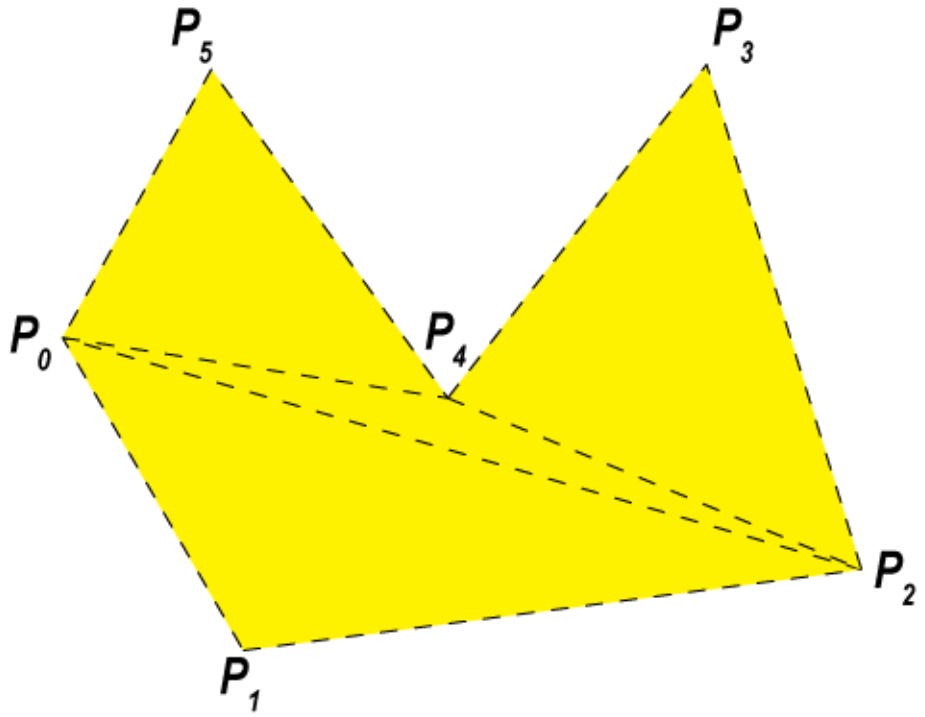
Algorithm

```
void Polygon::triangulation() {
  vector<Vector2D*> tmp;

  // copy the list of vertices into the tmp list
  for (int i=0; i<N; i++) tmp.push_back(&(tabPts[i]));

  tmp.push_back(&(tabPts[0])); // add 2 points for safety
  tmp.push_back(&(tabPts[1]));
  int n=N;
  while (n>2) { // while we can add a triangle to tabTriangles
    int i=0;
    auto p = tmp.begin();
    bool test;

    // create a triangle using p,p+1 and p+2 as vertices
    Triangle T(*p,*(p+1),*(p+2));
    do { // search a triangle without another points inside
      test=!T.isEmpty(tmp,i+3);
      if (test) {
        i++;
        p++; // update T to the next triangle
        T=Triangle(*p,*(p+1),*(p+2));
      }
    } while (i<n-2 && test);
    tabTriangles.push_back(T); // add T to tabTriangles
    tmp.erase(p+1); // remove point(p+1) from tmp
    n--;
  }
}
```

Compute the surface of a polygon

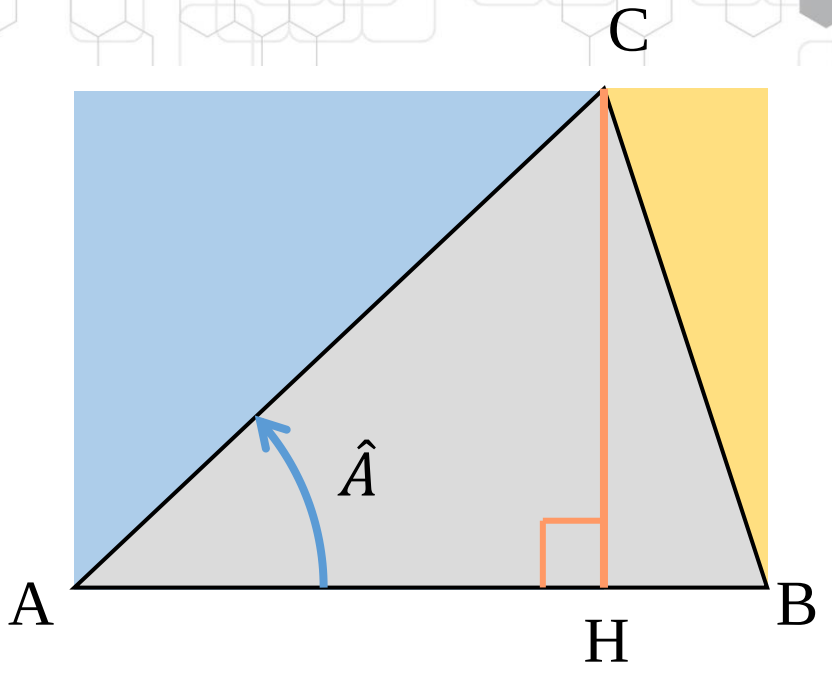
- How to compute the surface of a polygon
  1. Triangulate into $T = \{T_0, T_1, \cdots, T_{n-1}\}$
  2. Sum up the surfaces of the triangles

$$S = \sum_{i=0}^{n-1} S(T_i)$$



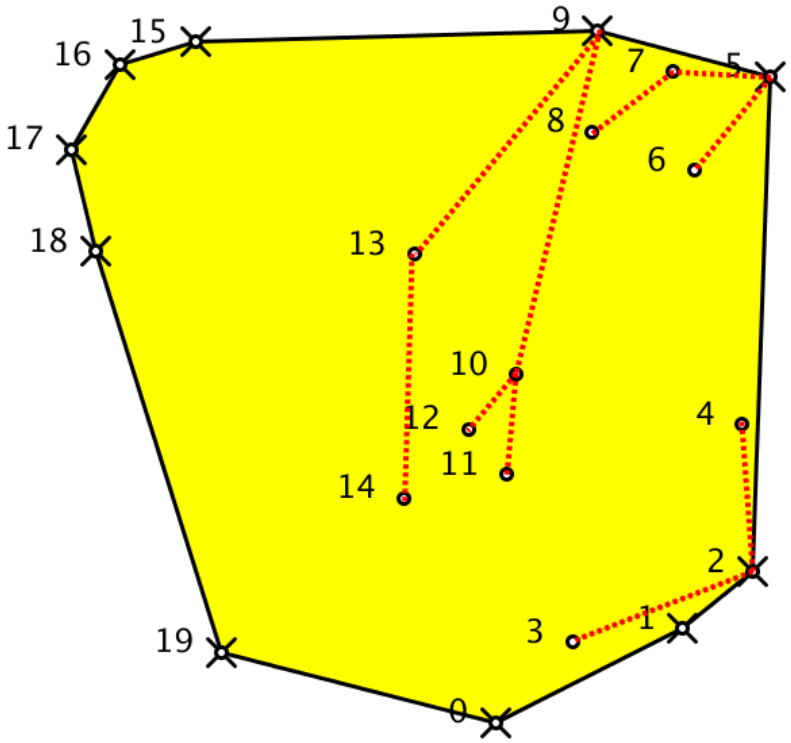
Compute the surface of simple triangle

- $S_{(ABC)} = S_{(AHC)} + S_{(HBC)}$
- $S_{(AHC)} = \frac{1}{2} AH \times HC$ and $S_{(HBC)} = \frac{1}{2} HB \times HC$
- $S_{(ABC)} = \frac{1}{2} (AH + HB) \times HC$
- $HC = \sin \hat{A} \times AC$
- $S_{(ABC)} = \frac{1}{2} \left(\sin \hat{A} \times AB \times AC \right)$
- $S_{(ABC)} = \frac{1}{2} \left[\overrightarrow{AB} \times \overrightarrow{AC} \right]_z$



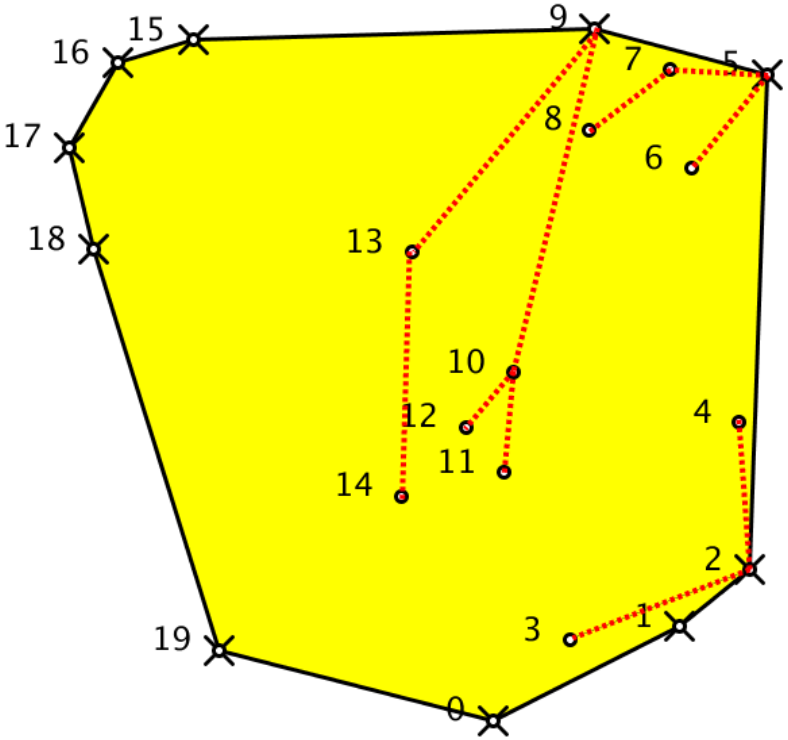
$$S_{(ABC)} = \frac{\overrightarrow{AB}_x \times \overrightarrow{AC}_y - \overrightarrow{AB}_y \times \overrightarrow{AC}_x}{2}$$

Convex hull



Convexe hull

- Definition:
  - The **convex hull** (or **convex envelop**) of a set of point P_i of $\mathcal{R}^2$ notated $Conv(P)$ is the smallest convex polygon S which admits every points P_i in its border or inside.



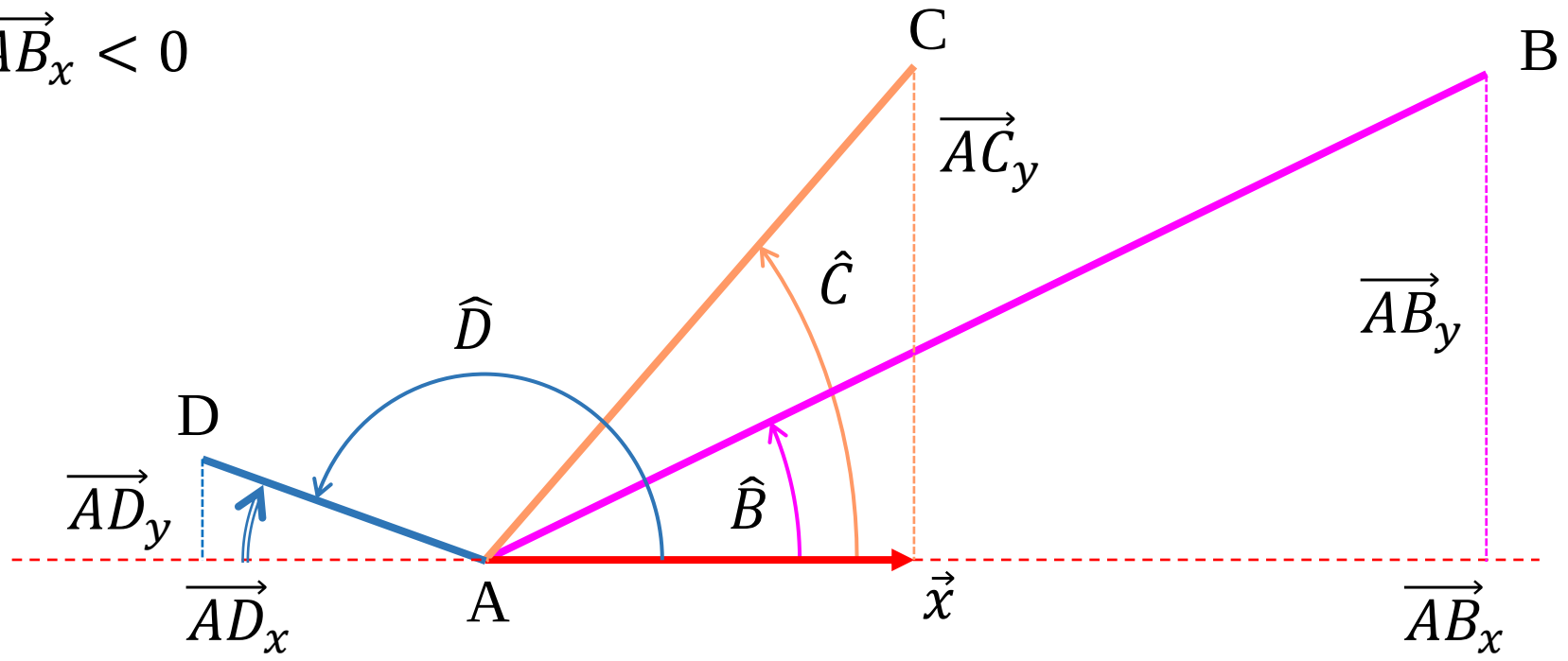
Graham scan algorithm

- Constructs Conv from a set of points

```
Let N the number of points
Let P the array of points
Let S a stack that will store the result

Swap P_0 with the point with the lowest y-coordinate
Sort points P_i | i ∈ [1..N] by ascending polar angle (P_0 P_i, x)
S.push({P_0, P_1, P_2})
for i = 3 to N-1 do
  while (CCW(S.top(-1), S.top(), P_i) <= 0) ) do
    S.pop()
  done
  S.push(P_i)
done
Create Conv using S points
```

Ascending polar angle

$$\hat{B} = \begin{cases} \sin^{-1}\left(\dfrac{\overrightarrow{AB}_y}{AB}\right) & \overrightarrow{AB}_x \geq 0 \\ \pi - \sin^{-1}\left(\dfrac{\overrightarrow{AB}_y}{AB}\right) & \overrightarrow{AB}_x < 0 \end{cases}$$



Implementation in C++

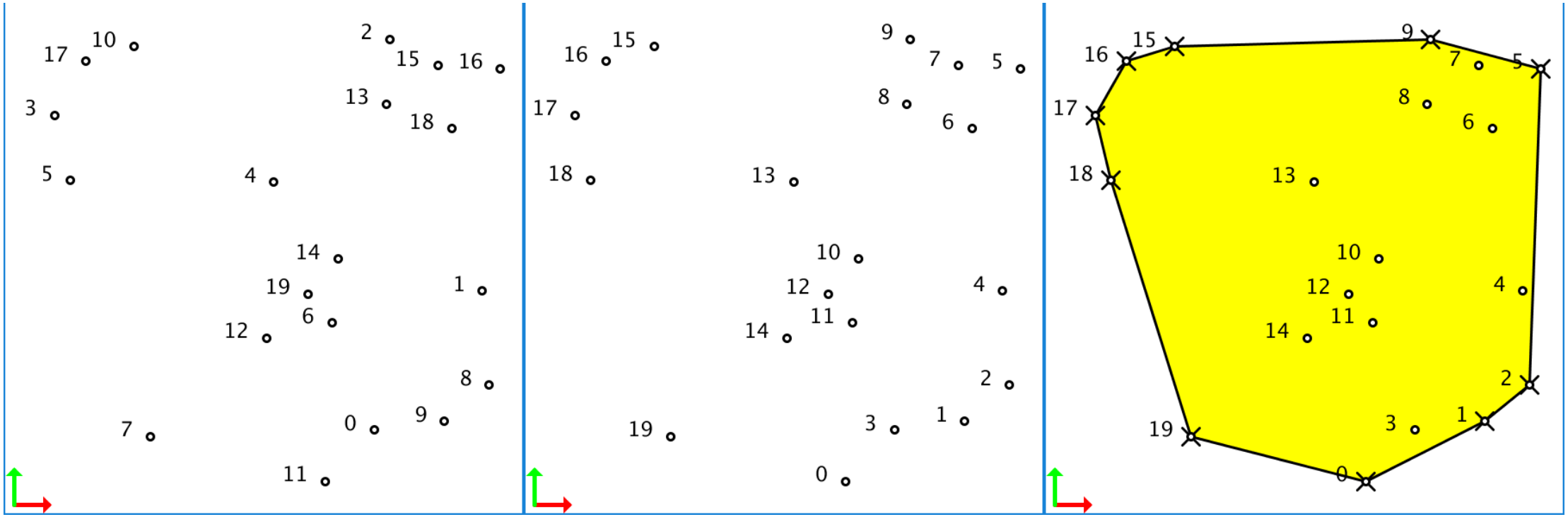
```
bool polarComparison(Vector2D P1,Vector2D P2) {
        double a1 = asin(P1.y/sqrt(P1.x*P1.x+P1.y*P1.y));
        if (P1.x<0.0) a1=M_PI-a1;
        double a2 = asin(P2.y/sqrt(P2.x*P2.x+P2.y*P2.y));
        if (P2.x<0.0) a2=M_PI-a2;
        return a1<a2;
}

Vector2D origin(points.begin()->x,points.begin()->y);
// copy points in a set of points relative to points[0]
QVector<Vector2D> pointsRelative;
for (auto pOrig:points) {
        pointsRelative.push_back(Vector2D(pOrig.x-origin.x,pOrig.y-origin.y));
}
// sorting point with angular criteria
std::sort(pointsRelative.begin()+1, pointsRelative.end(),polarComparison);
```

Exemple

Set of points P

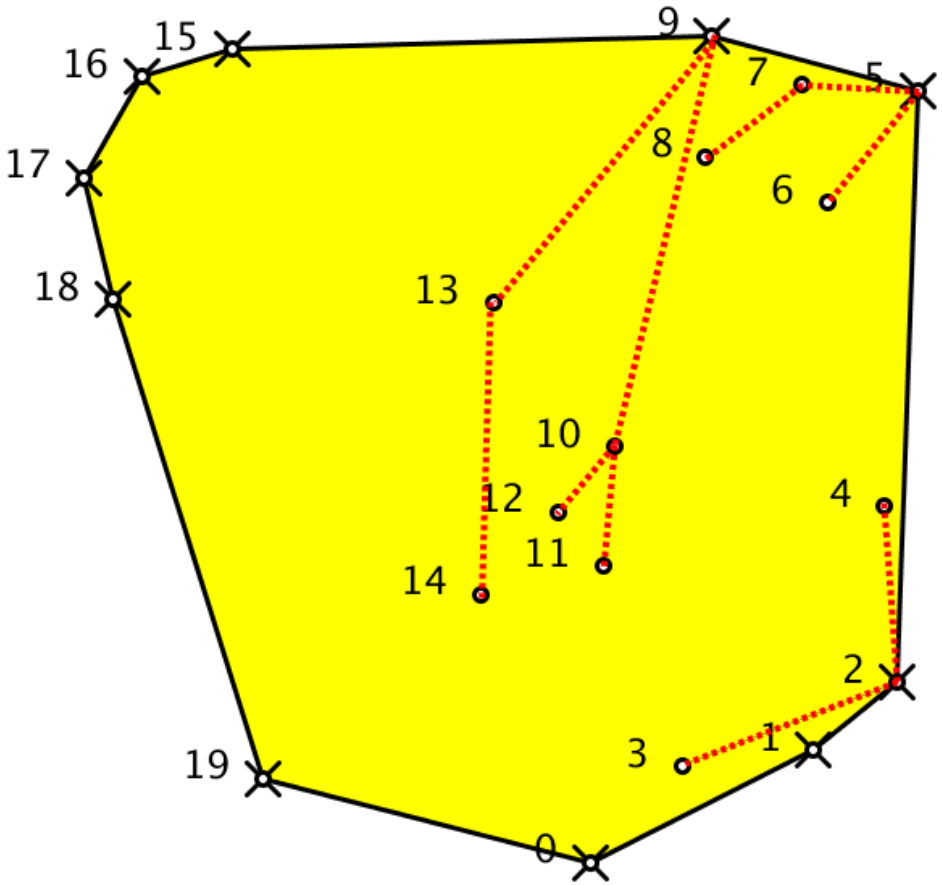
Sorting by ascending polar angle

Conv(P) envelop



Algorithm steps

```
S.push({P_0, P_1, P_2})
for i = 3 to N-1 do
  while (CCW(S.top(-1), S.top(), P_i) <= 0) ) do
    S.pop()
  done
  S.push(P_i)
done
Create Conv using S points
```

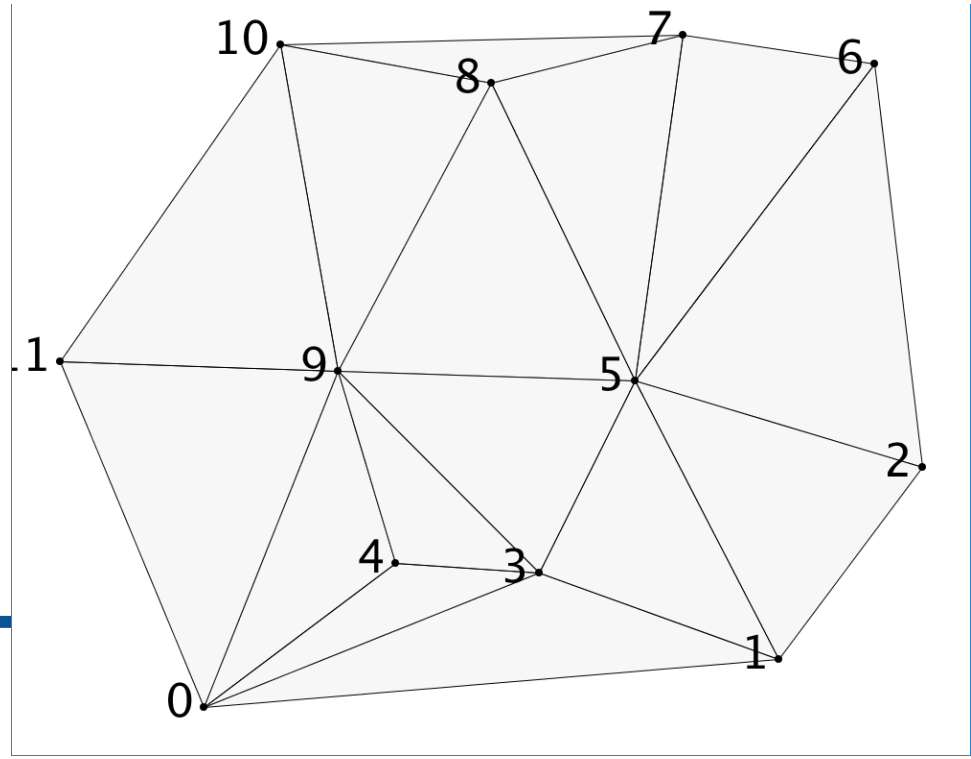


C++ Implementation

```
MyPolygon::MyPolygon(vector<Vector2D> &points) {
  assert(points.size()>3);
  auto p=points.begin();
  auto pymin=points.begin();
  // find point with minimal y and swap with first point
  while (p!=points.end()) {
    if (p->y<pymin->y) pymin=p;
    p++;
  }
  if (pymin!=points.begin()) iter_swap(points.begin(), pymin); // swap

  Vector2D origin(points.begin()->x,points.begin()->y);
  // copy points in a set of points relative to points[0]
  vector<Vector2D> pointsRelative;
  for (auto pO:points) {
      pointsRelative.push_back(Vector2D(pO.x-origin.x,pO.y-origin.y));
  }
  // sorting point with angular criteria
  sort(pointsRelative.begin()+1,pointsRelative.end(),polarComparison);

  stack<Vector2D*> CHstack;
  Vector2D *top_1,*top;
  CHstack.push(&pointsRelative[0]);
  CHstack.push(&pointsRelative[1]);
  CHstack.push(&pointsRelative[2]);
```

```
  vector<Vector2D>::iterator pi=pointsRelative.begin()+3;
  while (pi!=pointsRelative.end()) {
    top = CHstack.top(); // extract top and top_1
    CHstack.pop();       // from the stack
    top_1 = CHstack.top();
    CHstack.push(top);
    while (!isOnTheLeft(&(*pi),top_1,top)) {
      CHstack.pop(); // update top and top_1
      top = CHstack.top();
      CHstack.pop();
      top_1 = CHstack.top();
      CHstack.push(top);
    }
    CHstack.push(&(*pi));
    pi++;
  }
  // get stack points to create current polygon
  N=CHstack.size();
  Nmax = N;
  tabPts = new Vector2D[Nmax+1];
  int i=N-1;
  while (!CHstack.empty()) {
    tabPts[i--]=*(CHstack.top())+origin; // place the popped point in
    CHstack.pop();                       // the global referential
  }
  tabPts[N]=tabPts[0];
}
```

Delaunay triangulation

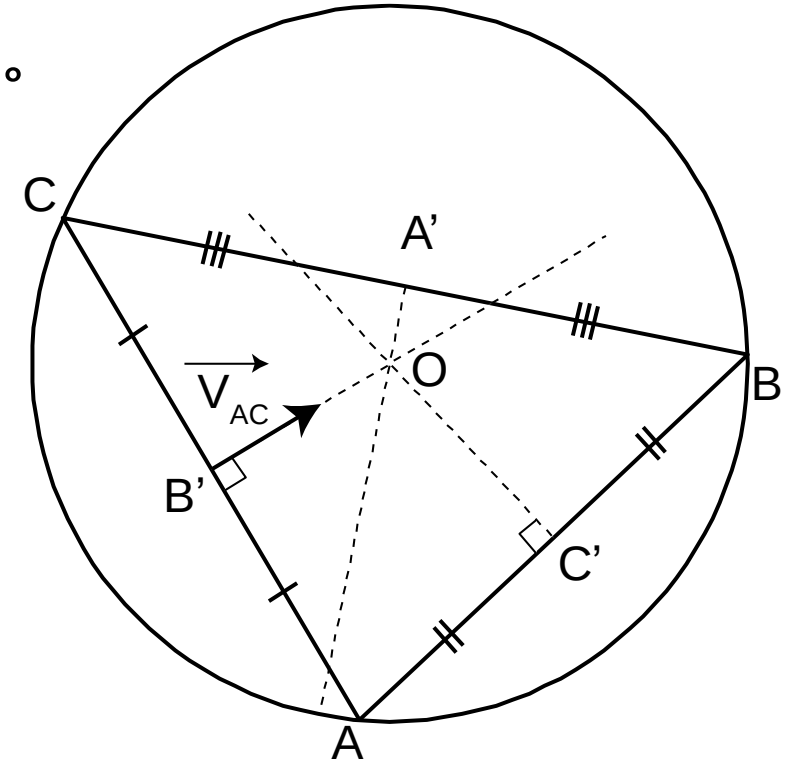


Delaunay triangulation

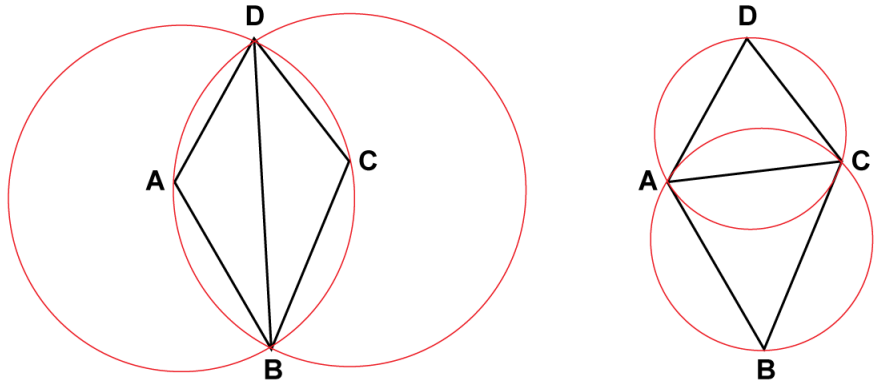
- Definitions
  - The **triangulation** of a set of points consists in building a set of disjoint triangles which fully **cover the convex hull** of the set of points. This set of triangles defines a **mesh**.
  - One important problem is to find a mesh as regular as possible, i.e. made of large triangles, which shape is close to equilateral triangle.
  - Delaunay's criteria:
    The **Delaunay triangulation** (also known as a Delone triangulation) for a given set P of discrete points in a plane is a triangulation DT(P) such that **no point in P is inside the circumcircle of any triangle** in DT(P).

Circumscribed circle or Circumcircle

- The circumcenter of a triangle is the intersection of the 3 perpendicular bisectors.
  - The line which cut an edge into two equal parts at 90°



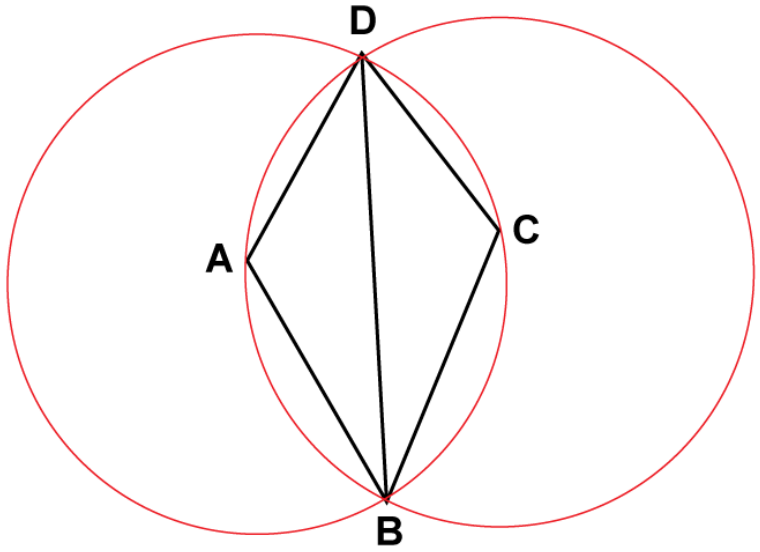
Delaunay triangulation

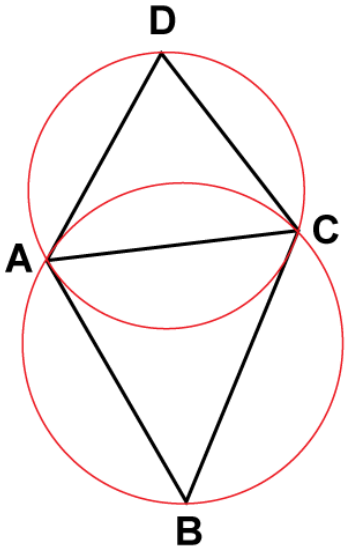


- Theorems
  - Let N be the number of points of P. **The union of every triangles** of a triangulation makes the **convex hull of the points**.
  - In the $\mathcal{R}^2$ plane, the triangulation of Delaunay is made of $O(N)$ triangles, if the convex hull is made of b vertices, each possible triangulation admits exactly $2N - 2 - b$ triangles.
  - If two points P_i and P_j are on the border of a circle C and there is **no other point** P_k inside C, then the edge $[P_iP_j]$ is part of the Delaunay triangulation.
  - "Opposite angle propriety": Considering two given triangles ABC and ACD, the sum of the angles $\hat{B}$ and $\hat{D}$ is less or equal to 180° if and only if these triangles follows Delaunay's criteria.

Delaunay triangulation algorithm

- Delaunay triangulation algorithm :
  - If two triangles ABD and BCD do not respect the Delaunay's criteria
  - We flip the common edge $[BD]$ to $[AC]$ in order to replace the two previous triangles by ABC and ACD which respect the Delaunay's criteria.





Check if a point D is in a circumcircle of ABC

- Compute the center O and the radius r and compare OD to r

$$\overrightarrow{AO} = \overrightarrow{AB'} + \overrightarrow{B'O}$$

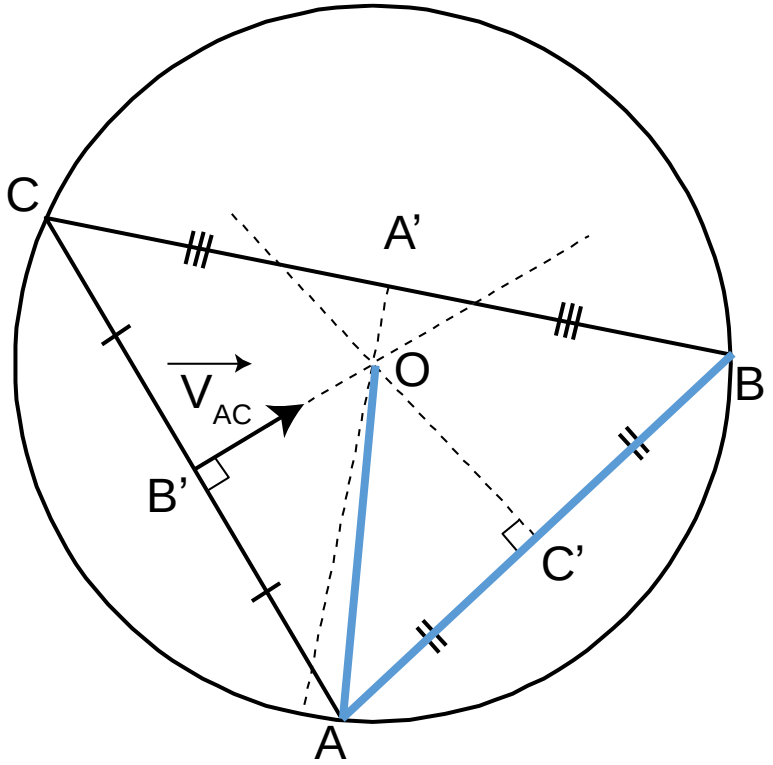
$$\overrightarrow{B'O} = k \times \overrightarrow{V_{AC}} = k \times ortho(AC)$$

$$\overrightarrow{AO} \cdot \overrightarrow{AB} = \left(\overrightarrow{AB'} + \overrightarrow{B'O}\right) \cdot \overrightarrow{AB}$$

$$\overrightarrow{AO} \cdot \overrightarrow{AB} = AO \times AB \times \cos(\hat{A}) = AC \times AB = \frac{1}{2} AB^2$$

$$\overrightarrow{AO} \cdot \overrightarrow{AB} = \left(\frac{1}{2}\overrightarrow{AC} + k \times \overrightarrow{V_{AC}}\right) \cdot \overrightarrow{AB} = \frac{1}{2} AB^2$$

$$k = \frac{AB^2 - \overrightarrow{AC} \cdot \overrightarrow{AB}}{2 \times \overrightarrow{V_{AC}} \cdot \overrightarrow{AB}}$$



k is the abscissa of O in $[B'\overrightarrow{V_{AC}})$

Check if a point D is in a circumcircle of ABC

- Theorem:
  - In $\mathbb{R}^2$, the point D is inside the circumcircle of the triangle ABC if the determinant of the following matrix is positive.

$$\det\begin{pmatrix} A_x & A_y & A_x^2+A_y^2 & 1 \\ B_x & B_y & B_x^2+B_y^2 & 1 \\ C_x & C_y & C_x^2+C_y^2 & 1 \\ D_x & D_y & D_x^2+D_y^2 & 1 \end{pmatrix} > 0 \quad \Leftrightarrow \quad \det\begin{pmatrix} A_x-D_x & A_y-D_y & A_x^2-D_x^2+A_y^2-D_y^2 \\ B_x-D_x & B_y-D_y & B_x^2-D_x^2+B_y^2-D_y^2 \\ C_x-D_x & C_y-D_y & C_x^2-D_x^2+C_y^2-D_y^2 \end{pmatrix} > 0$$

About determinant of matrices

$$det\begin{pmatrix} a & c \\ b & d \end{pmatrix} = ad - bc$$

$$det\begin{pmatrix} a & d & g \\ b & e & h \\ c & f & i \end{pmatrix} = a \times det\begin{pmatrix} e & h \\ f & i \end{pmatrix} - b \times det\begin{pmatrix} d & g \\ f & i \end{pmatrix} + c \times det\begin{pmatrix} d & g \\ e & h \end{pmatrix}$$

Comparison of two methods

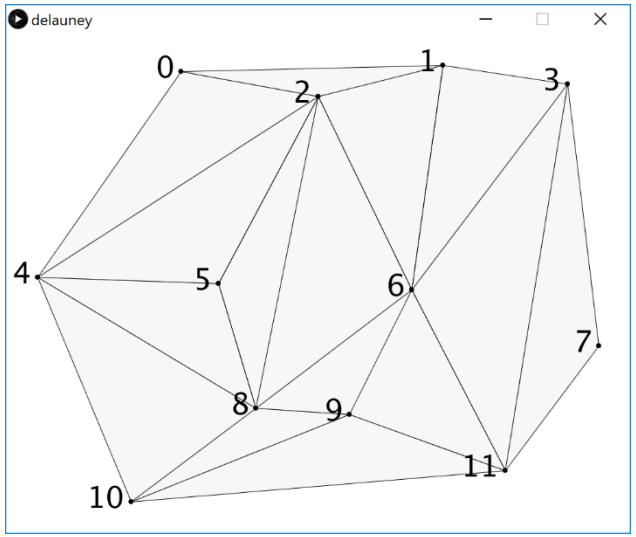
- Advantage of the second method
  - Is D is A,B or C

$$\det\begin{pmatrix} A_x - D_x & A_y - D_y & A_x^2 - D_x^2 + A_y^2 - D_y^2 \\ B_x - D_x & B_y - D_y & B_x^2 - D_x^2 + B_y^2 - D_y^2 \\ C_x - D_x & C_y - D_y & C_x^2 - D_x^2 + C_y^2 - D_y^2 \end{pmatrix}$$

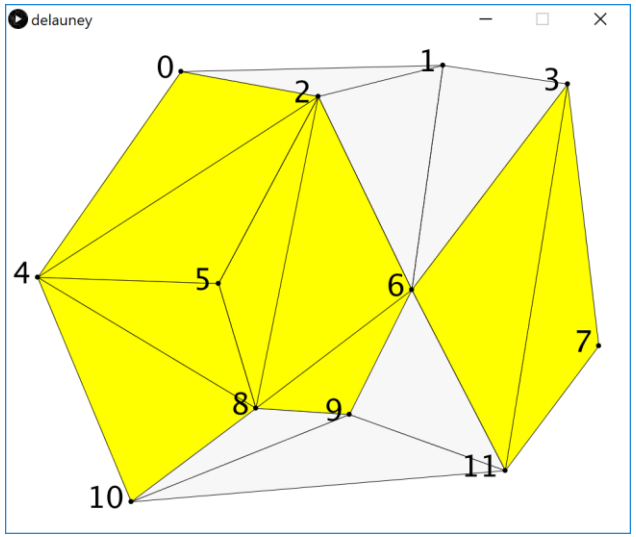
$$\det\begin{pmatrix} 0 & 0 & 0 \\ B_x - D_x & B_y - D_y & B_x^2 - D_x^2 + B_y^2 - D_y^2 \\ C_x - D_x & C_y - D_y & C_x^2 - D_x^2 + C_y^2 - D_y^2 \end{pmatrix} = 0$$

Delaunay triangulation algorithm

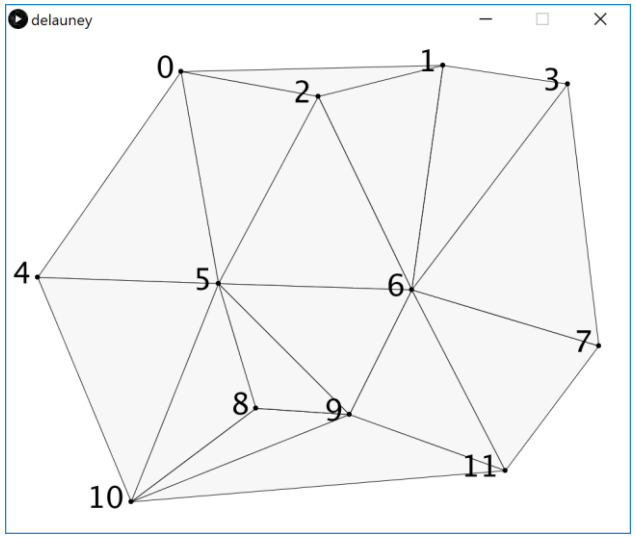
- Example on a given mesh:



Initial mesh

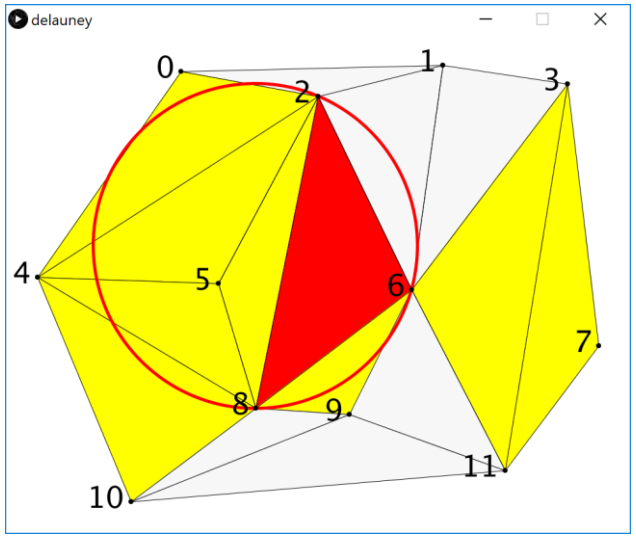


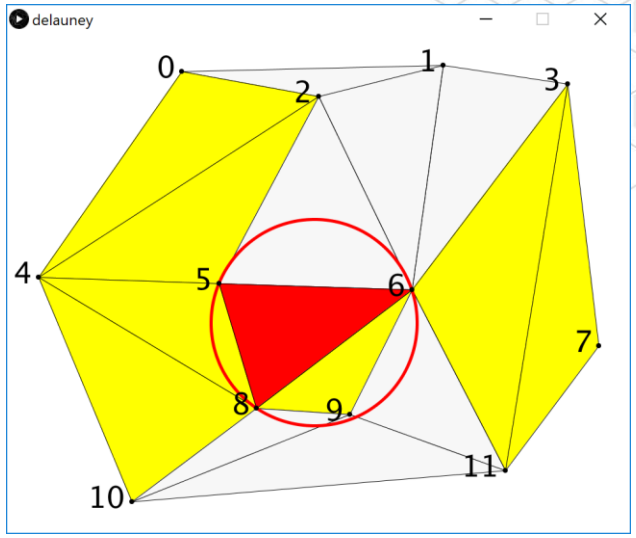
Non Delaunay triangles in yellow

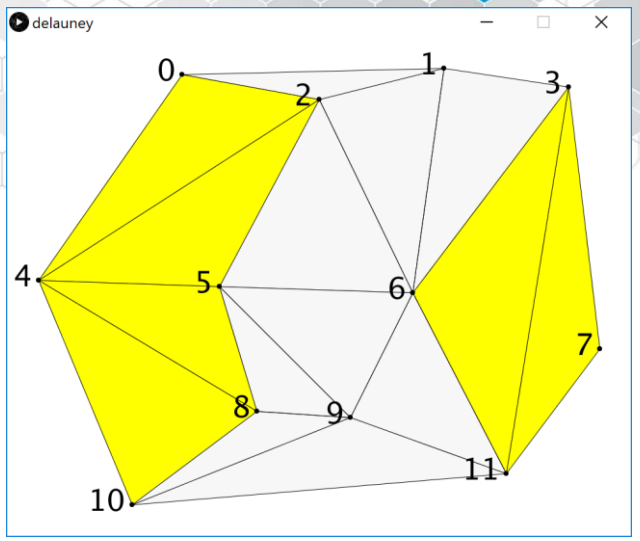


After flips

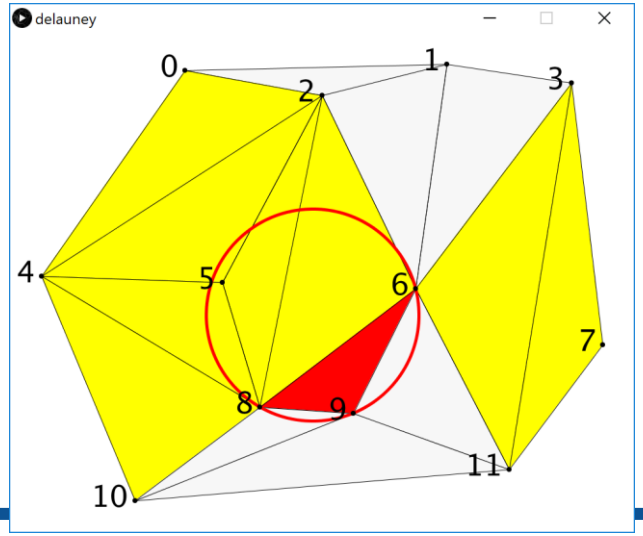
Flip orders!







- The flipping is impossible if the point which is inside the circumcircle is not a vertex of a neighbor triangle.
- This flipping of the triangle is postponed to the end of the treatments.

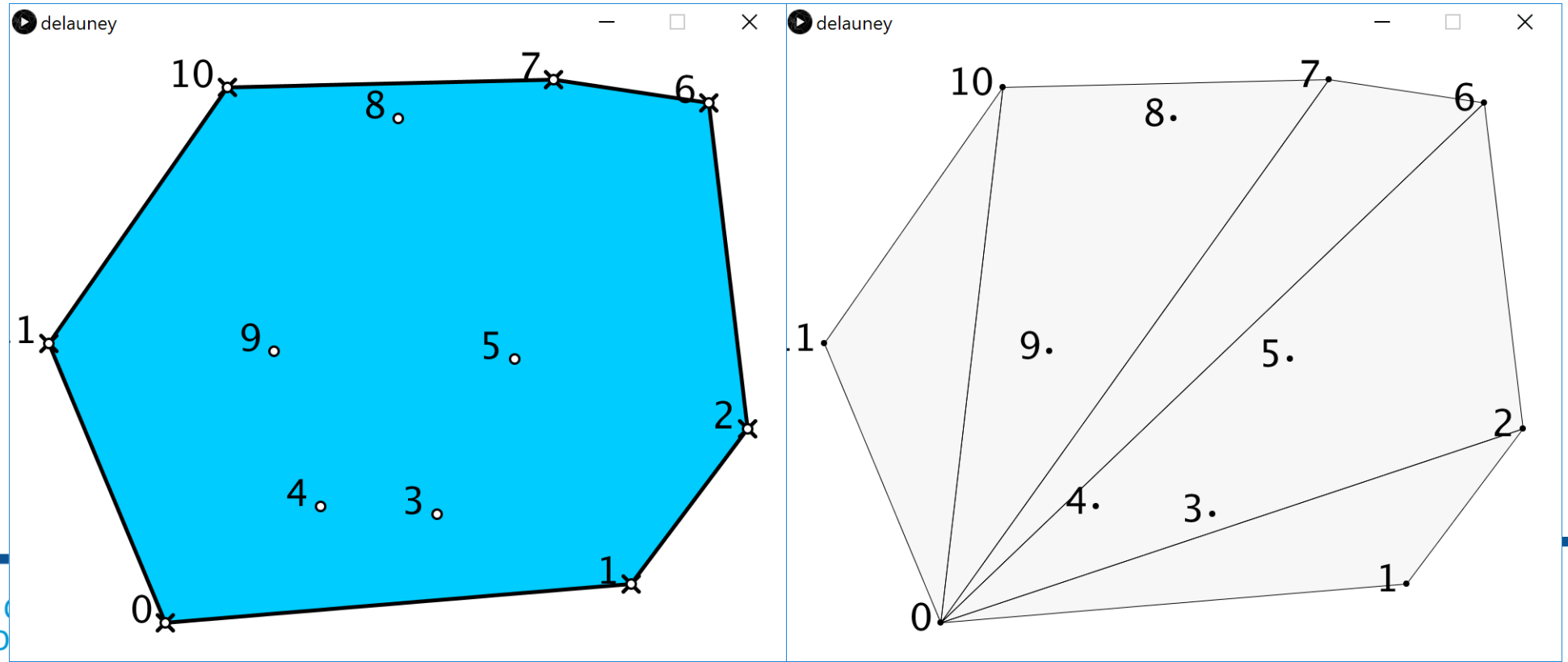


Algorithm

```
void Mesh::solveDelaunay() {
  list<Triangle*> processList;
  auto t = triangles.begin(); // copy tabTriangles in a list
  while (t!=triangles.end()) {
    processList.push_back(&(*t));
    t++;
  }

  while (processList.size()>1) { // while a triangle is in the list
    Triangle *current = processList.front(); // pop current
    processList.pop_front();
    if (!current->isDelaunay) { // if current is not Delaunay
      Triangle *Tneighbor = neighborInside(current);
      if (Tneighbor!=nullptr) { // and if a neighbor is available
        flip(current,Tneighbor); // flip the common edge
        // remove Tneighbor form the list
        auto tr=processList.begin();
        while (tr!=processList.end() && (*tr)!=Tneighbor) tr++;
        if (tr!=processList.end()) processList.erase(tr);
      } else {
        processList.push_back(current); // postpone the treatment
      }
    }
  }
}
```

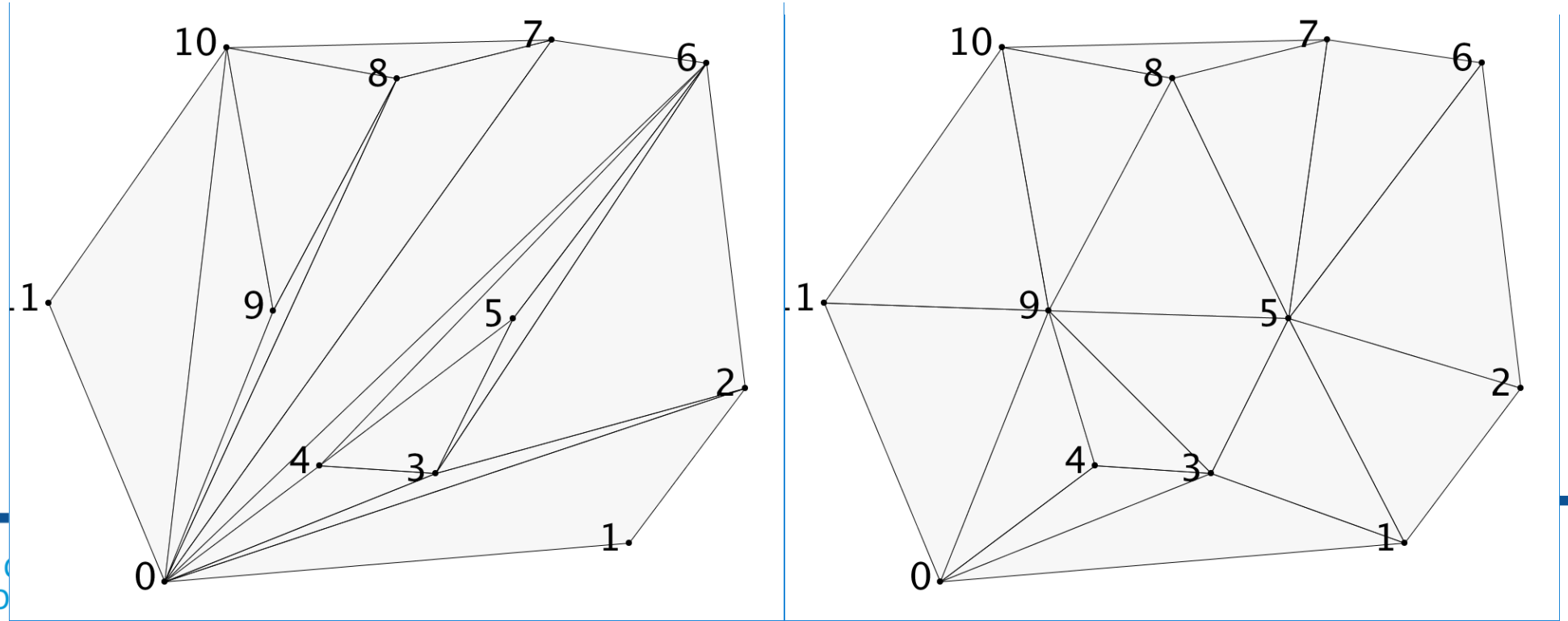
Creating a mesh from a set of points

1. Calculate **convex hull** of points $\mathrm{P}(P_0 \dots P_{N-1})$
2. **Triangulation** of the convex hull $\mathrm{E}(E_0 \dots E_{m-1})$,
   - Create a Triangle(E_0, E_i, E_{i+1}) $\forall i \in [1, m-2]$

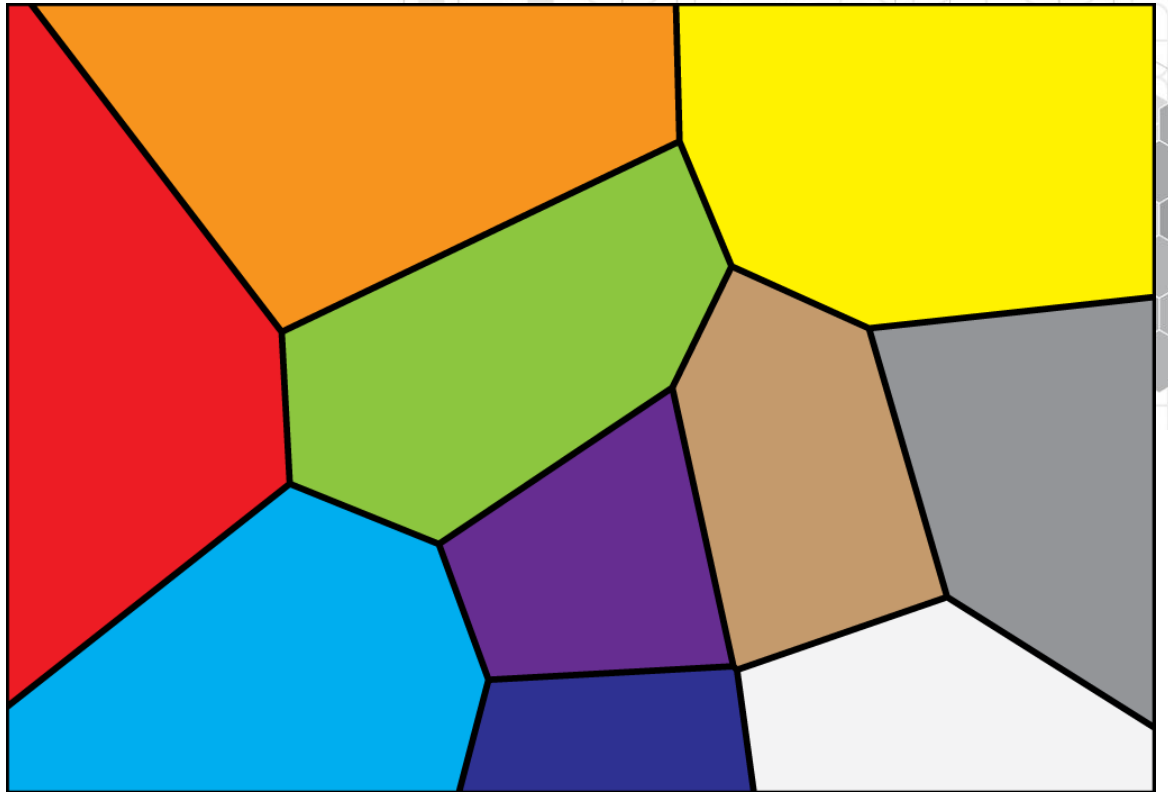


Creating a mesh from a set of points

3. • Add the **interior points**
   - If $P_i \in T(P_A, P_B, P_C)$ then
     - delete T
     - and add the triangles (P_A, P_B, P_i), (P_B, P_C, P_i) and (P_C, P_A, P_i)
4. • Apply the Delaunay triangulation algorithm.



Voronoi Diagram



Voronoi Diagram

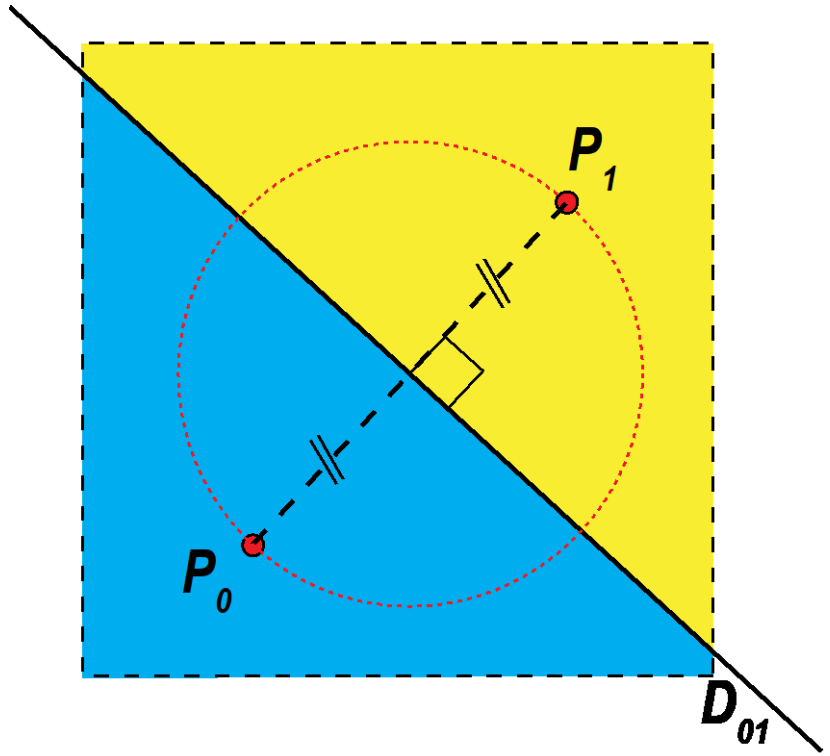
- Definitions
  - Given points $P = \{p_i \forall i \in [0 \cdots n-1]\}$, the Voronoi region of point p_i $V(p_i)$ is the set of points at least as close to p_i as to any other point in P.

$$V(p_i) = \{x | |p_i - x| \leq |p_j - x| \forall j \in [0 \cdots i-1]\}$$

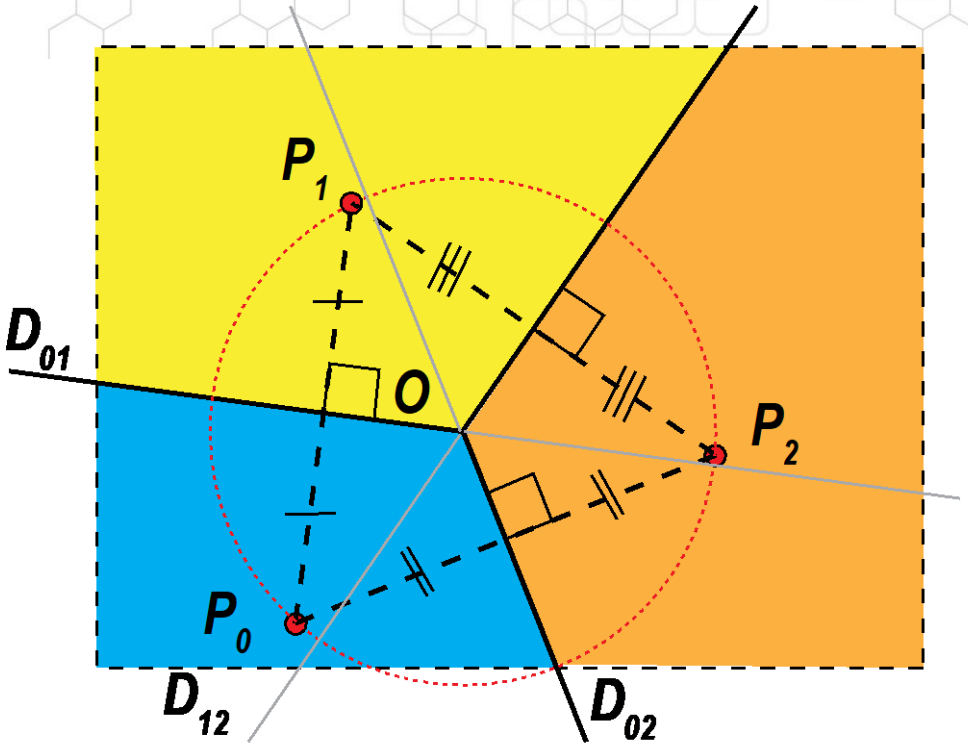
  - The Delaunay triangulation is the straight-line dual of the Voronoi Diagram.
    - Circumcenters of Delaunay's triangles are Voronoi diagram vertices.
    - Delaunay's vertices are centers of Voronoi regions.

Voronoi Diagram examples

2 points

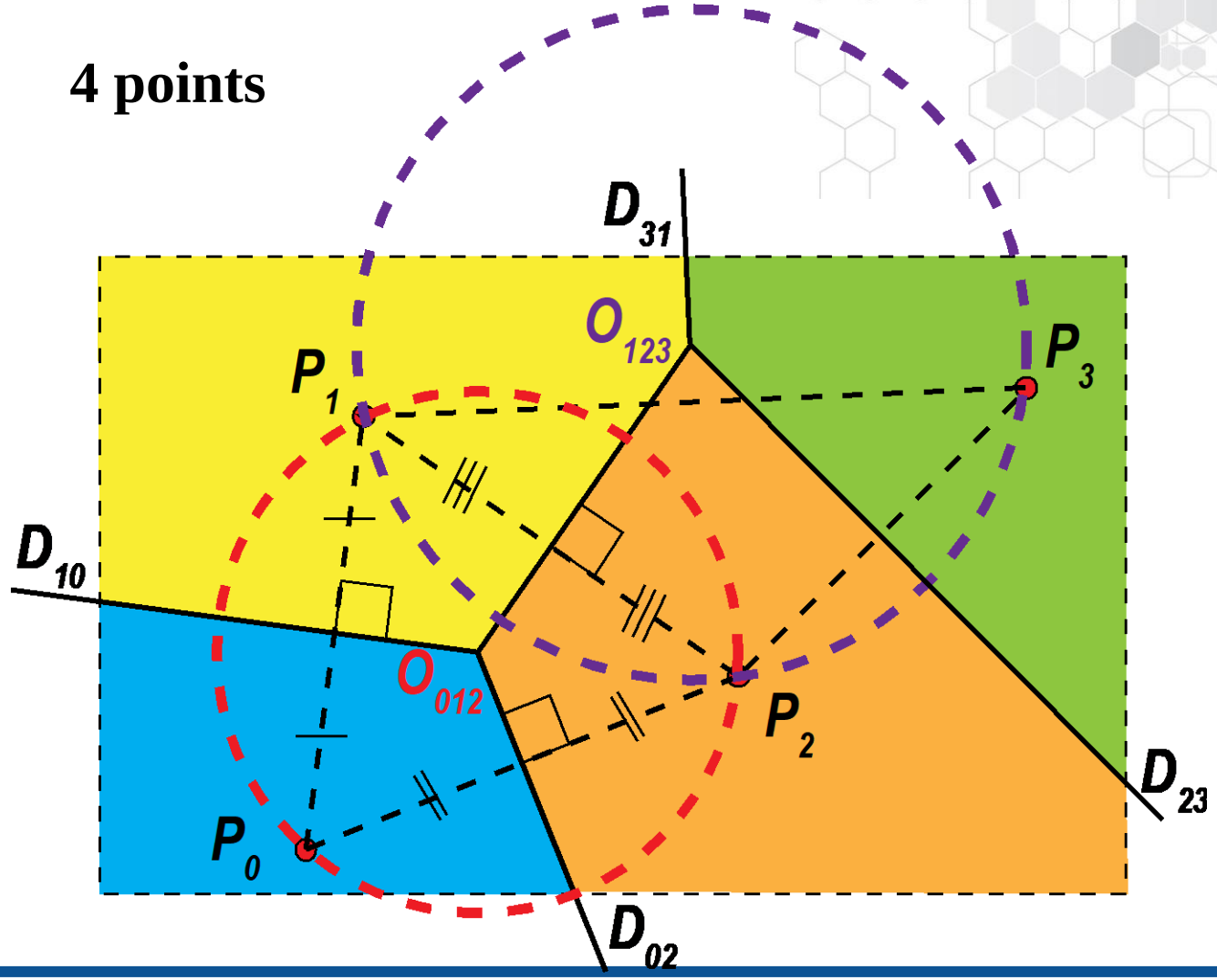


3 points

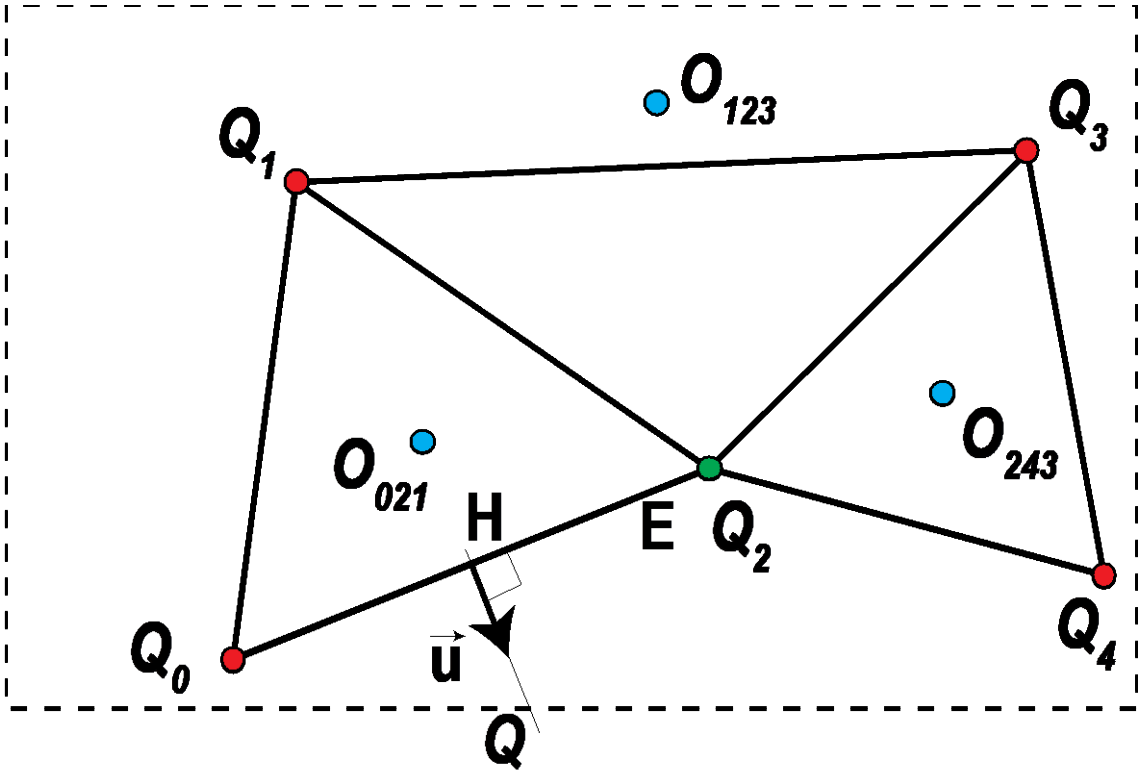


The regions are defined by **the perpendicular bisectors** of P_iP_j

Voronoi Diagram examples



Algorithm, from a Delaunay Mesh



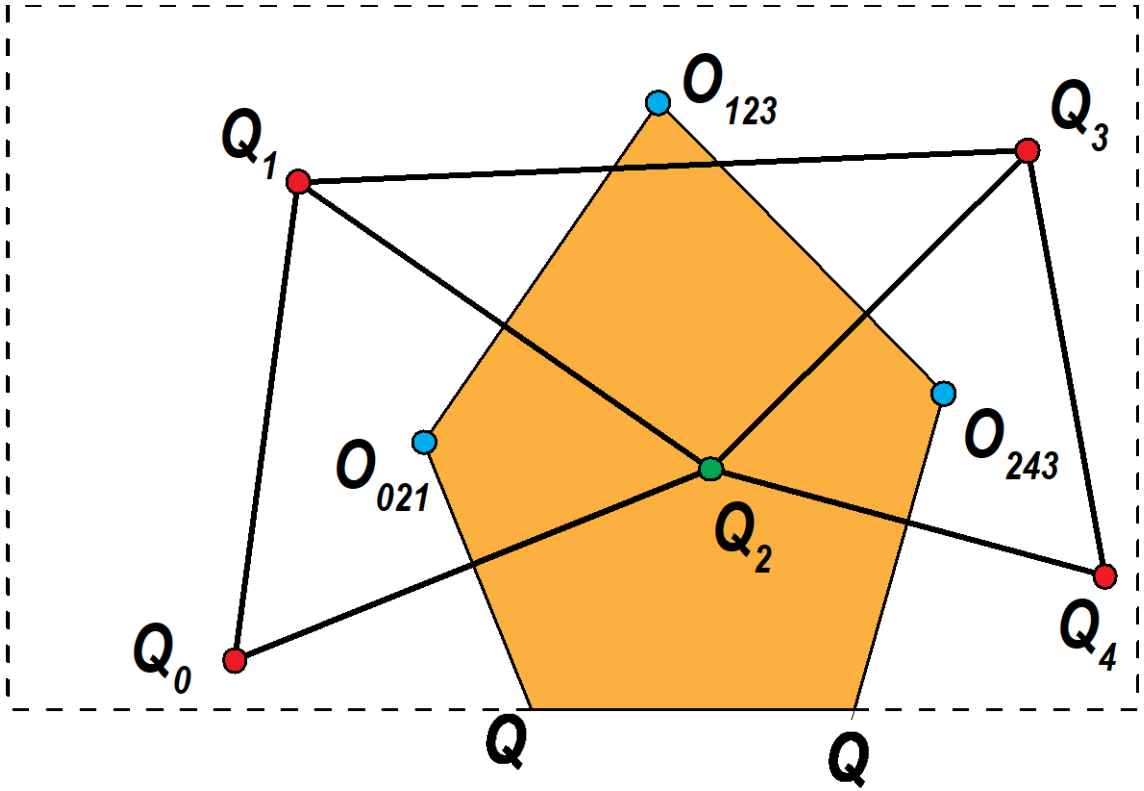
Considering Q_2

$T = \{Q_0Q_1Q_2, Q_1Q_3Q_2, Q_3Q_4Q_2\}$

$t = Q_0Q_1Q_2$

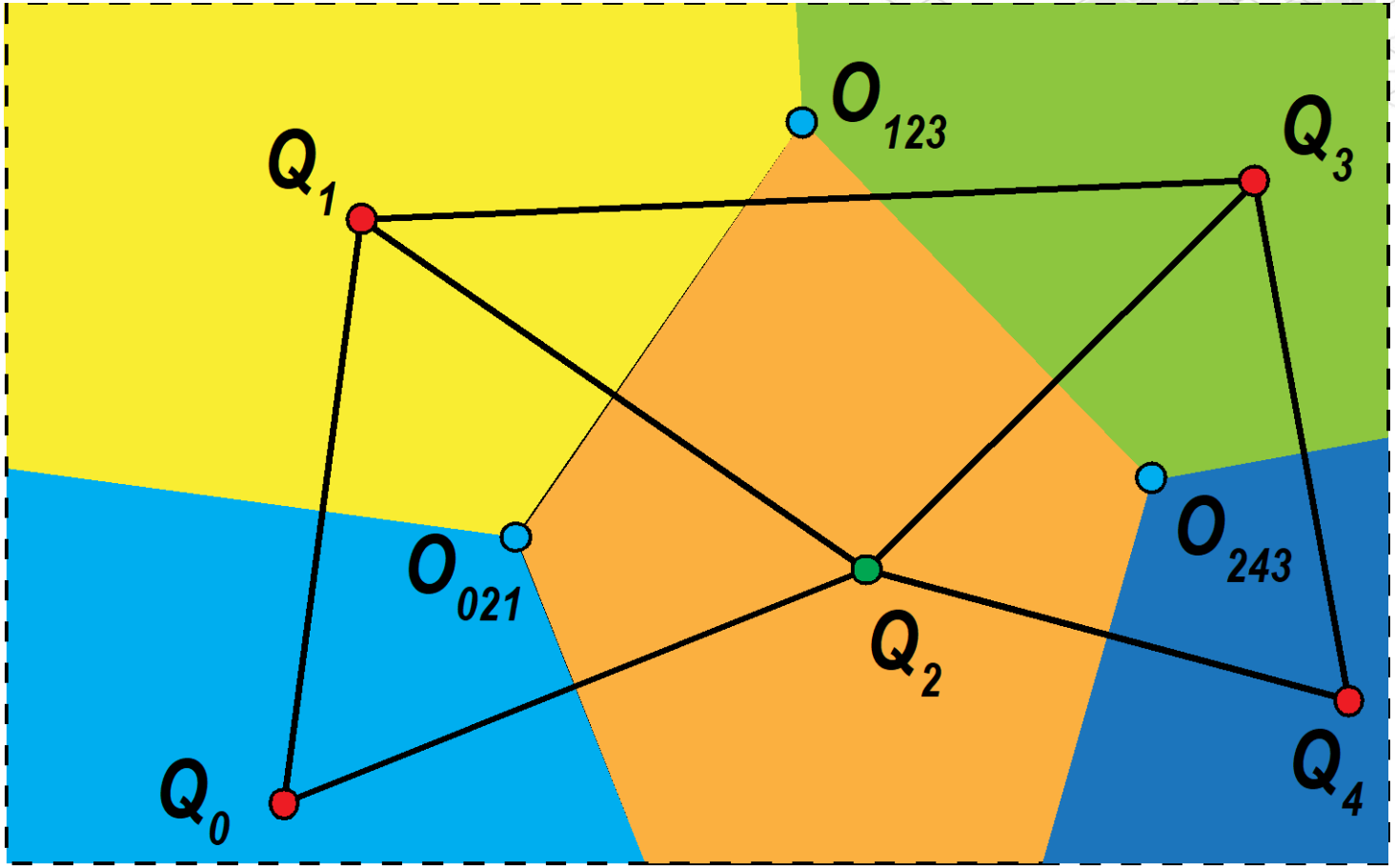
```
Data: D a Delaunay Mesh
Result: P a list of polygons
1  begin
2      foreach vertex Q_i of D do
3          Create a new polygon P_i
4          T ← subset of triangles t_i(a,b,c) of D where Q_i ∈ {a,b,c}
5          t ← leftTriangle(T)
6          isOpened ← (t exists)
7          if isOpened then
8              E ← nextEdge(t,Q_i)
9              H ← center of E
10             u⃗ ← rightOrthogonalVector(E)
11             Q ← IntersectionWithBorders(H,u⃗,0,0,width,height)
12             addPoint(Q,P_i)
13         else
14             t ← first(T)
15         end
```

Algorithm, from a Delaunay Mesh

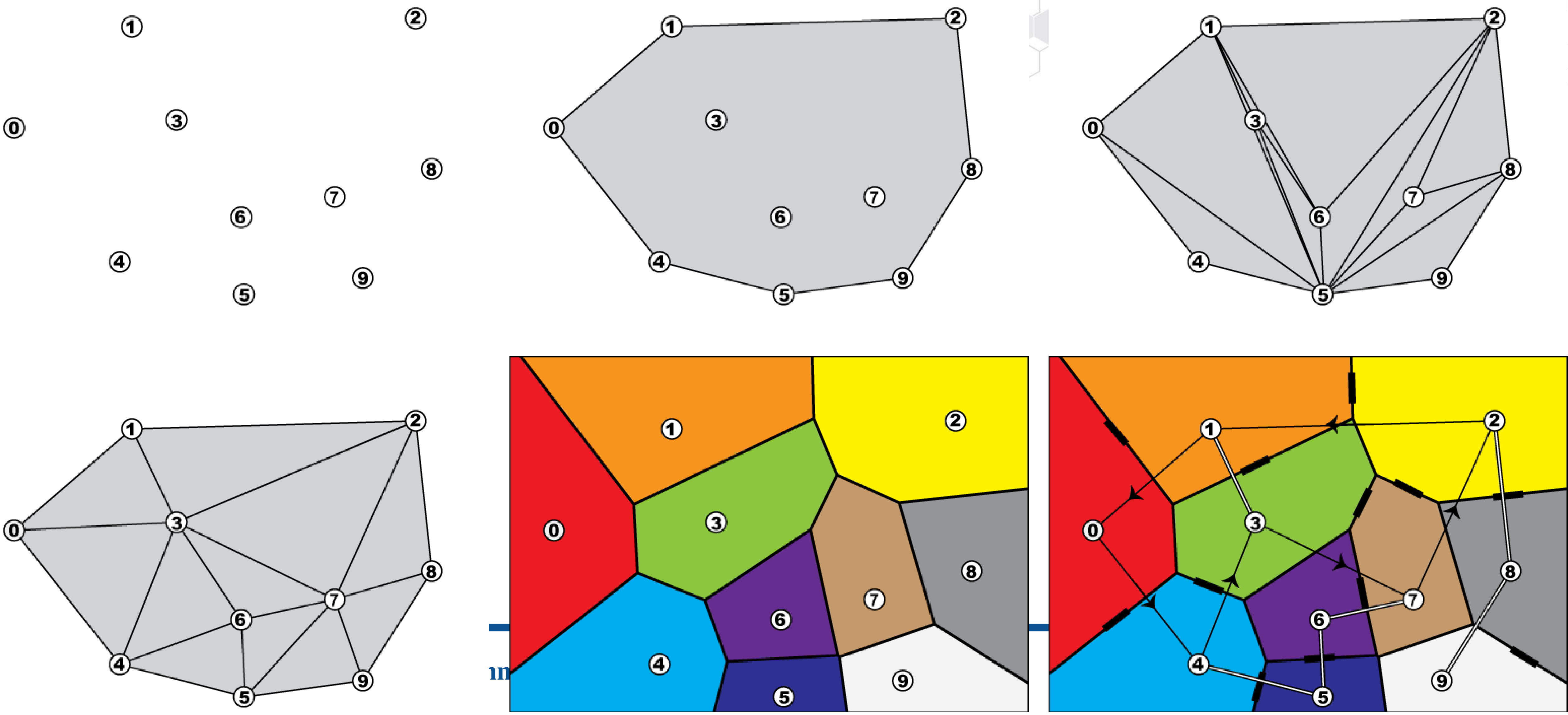



```
16      while size(T) >1 do
17          addPoint(circumcenter(t),P_i)
18          t_prev ← t
19          t ← rightNeighbor(t,Q_i)
20          removeTriangle(t_prev,T)
21      end
22      addPoint(circumcenter(t),P_i)
23      if isOpened then
24          E ← prevEdge(t,Q_i)
25          H ← center of E
26          u⃗ ← rightOrthogonalVector(E)
27          Q ← IntersectionWithBorders(H,u⃗,0,0,width,height)
28          addPoint(Q,P_i)
29      end
30      removeTriangle(t,T)
31      addCornerPoints(P_i,0,0,width,height)
32      push(P_i,P)
33  end
34 end
```

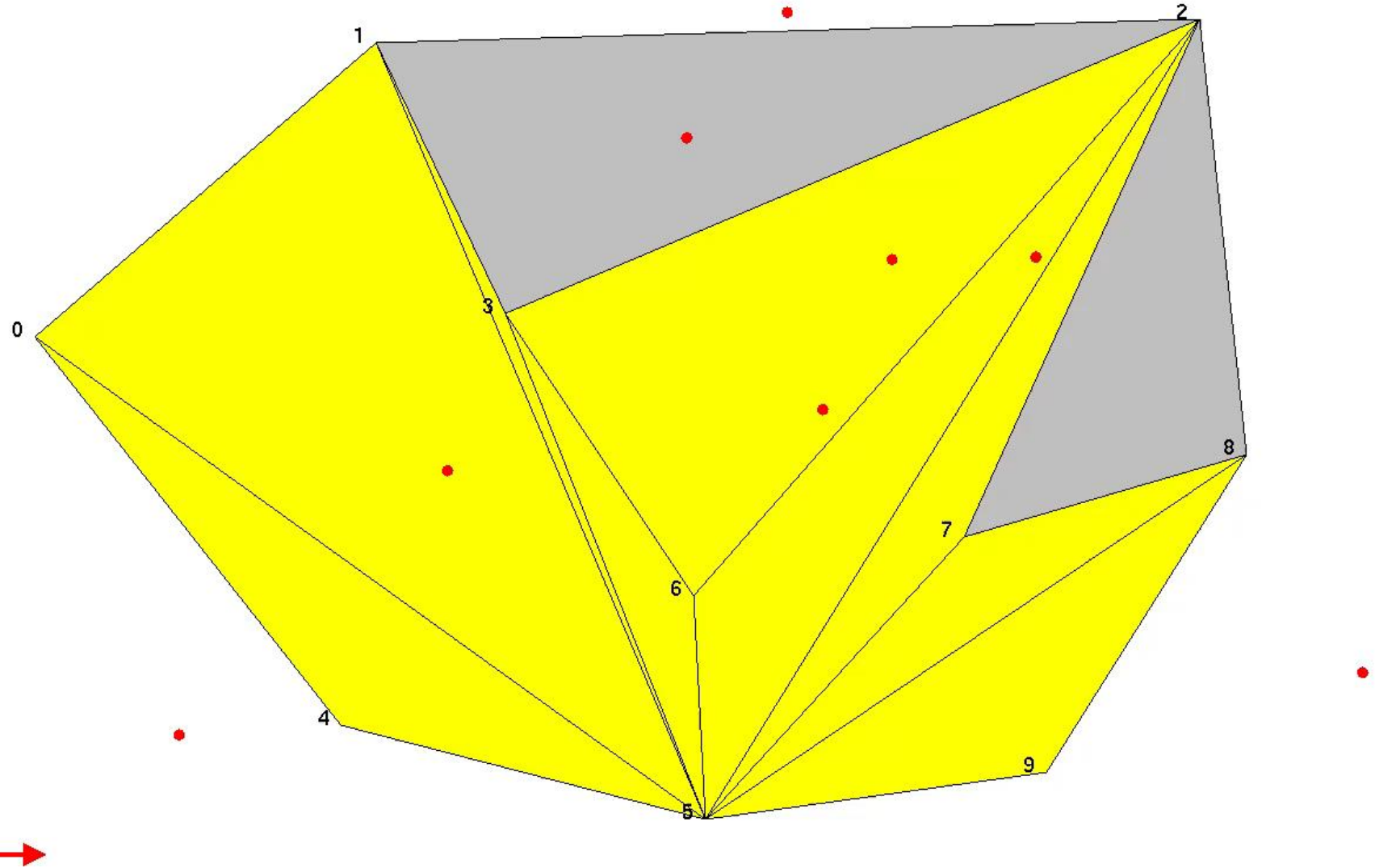
Algorithm, from a Delaunay Mesh



Result on an other configuration



An application: background of a simulation game



femto-st SCIENCES & TECHNOLOGIES

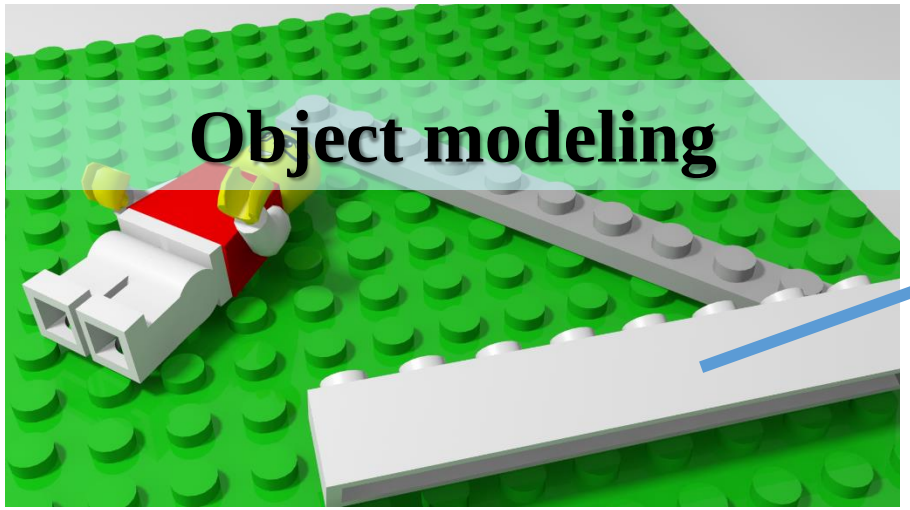
3D algorithms
Initiation to image synthesis

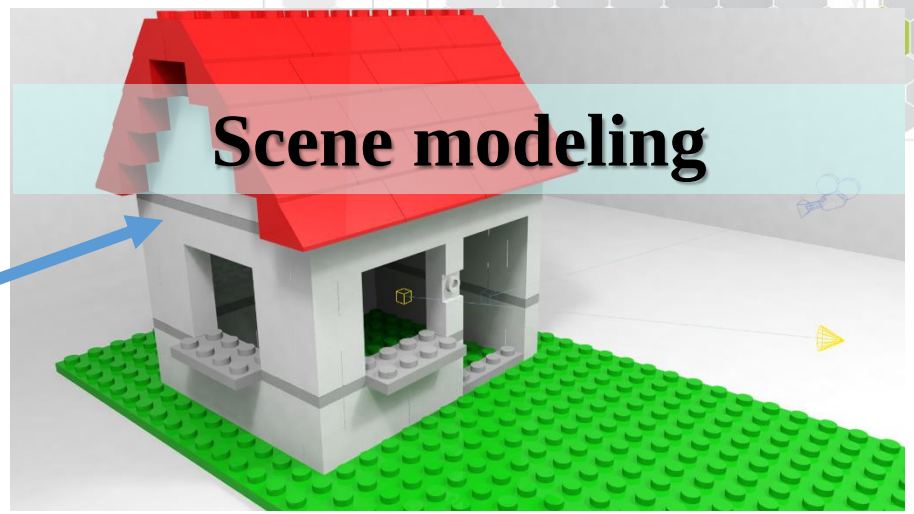
Examples with OpenGL

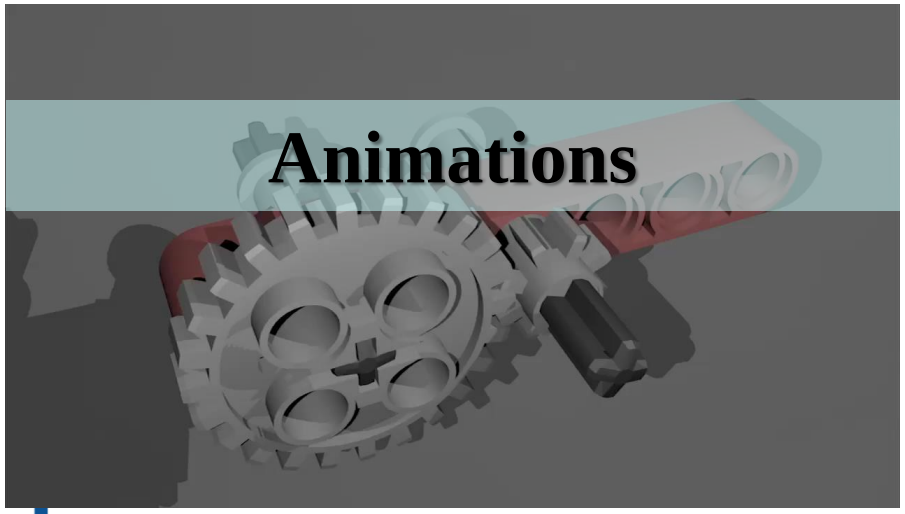
Image synthesis workflow

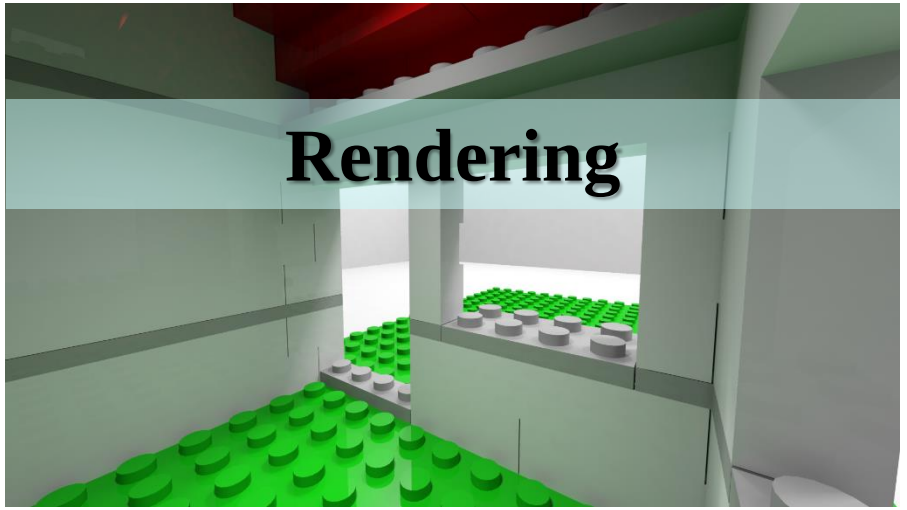
- Goal: create an image
  - Geometric modeling
    - The object
    - The scene
  - Lightning
    - Light ⇔ materials interactions
  - Rendering algorithms
    - ZBuffer
    - Ray-tracing

From the model to the image

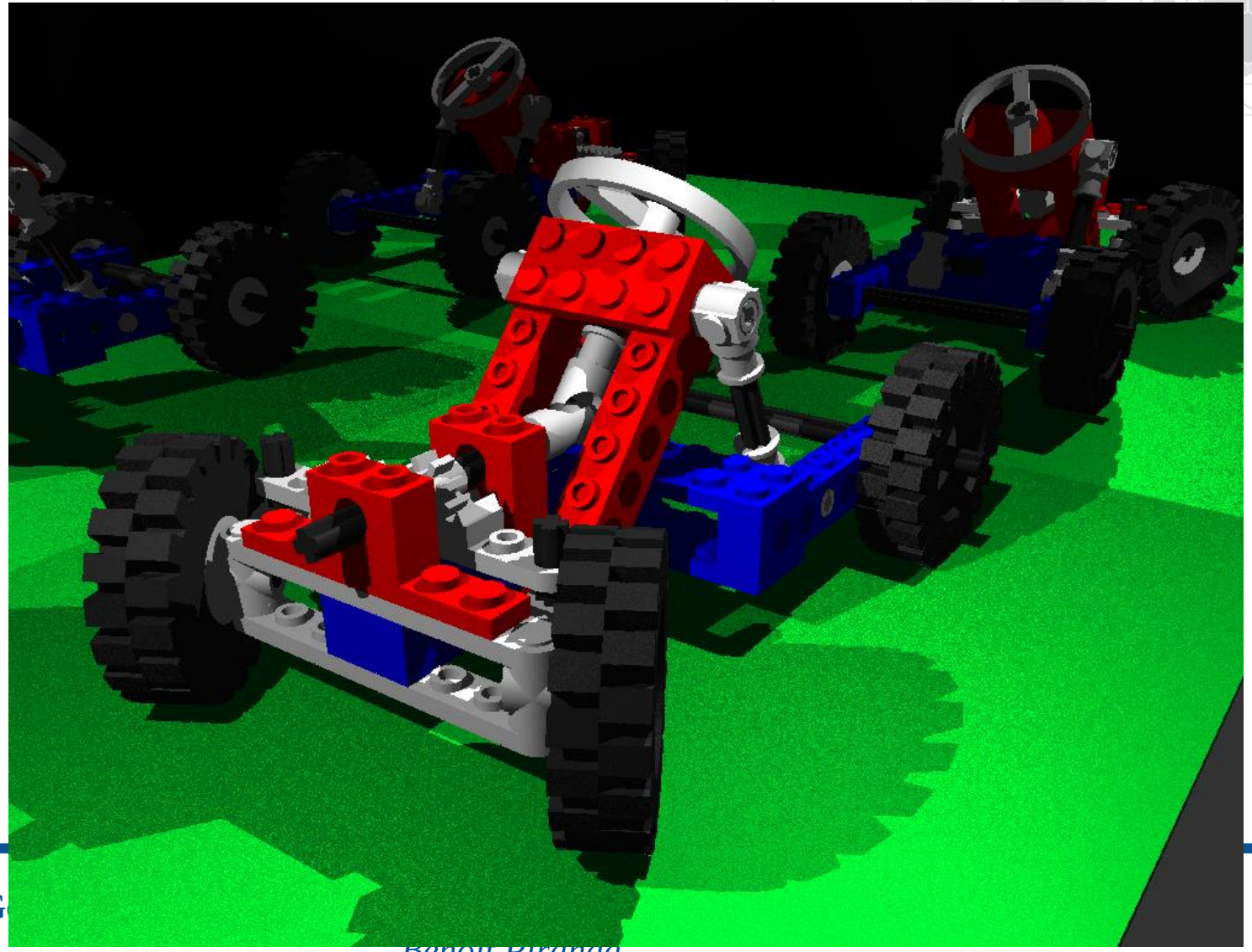








Using Constructive Solid Geometry Tree



Geometry modeling

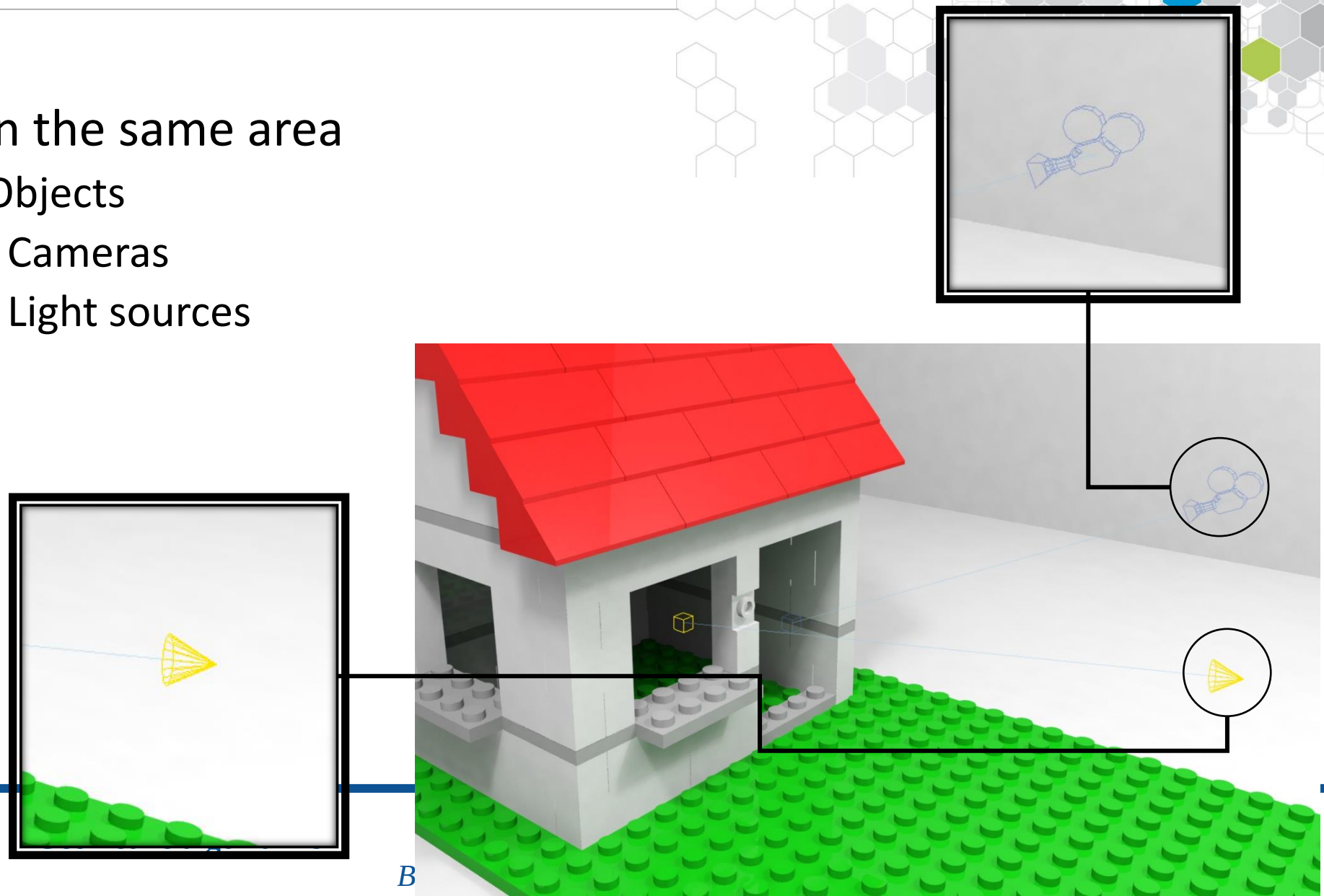
- Goal
  - Describe the shape of objects (local referential)
  - Place objects in the scene (global referential)
  - Tools participating to the creation of the images
    - Cameras (points of view, direction, view angle)
    - Light sources (position, color, power…)
- Geometrical operations
  - Transform the objects coordinates into the camera coordinate system
  - Project objects on the screen

Basic OpenGL 3D functions

- OpenGL drawing tools
  - Basic 3D shapes:
    - GL_TRIANGLE (3N vertices),
    - GL_QUADS (4N verticies),
    - GL_POLYGON (N vertices)
- GLUT Objects
  - glutSolidCube(size)
  - glutSolidSphere(radius,slice,stacks)
  - glutSolidTorus(innerRadius,outerRadius,nsides,rings)
  - glutSolidCone(base,height,slice,stacks)
  - glutSolidTeapot(size)

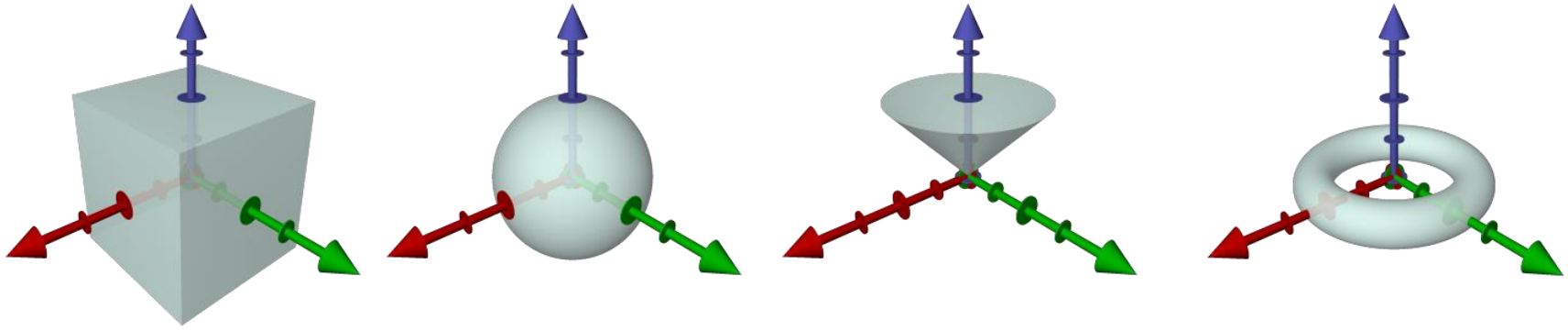
Scene modeling

- Place in the same area
  - 3D Objects
  - Add Cameras
  - Add Light sources
  - …



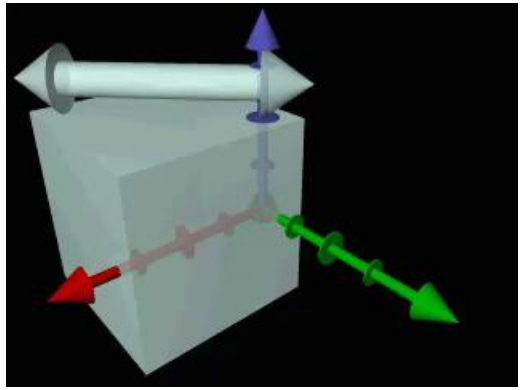
Scene modeling

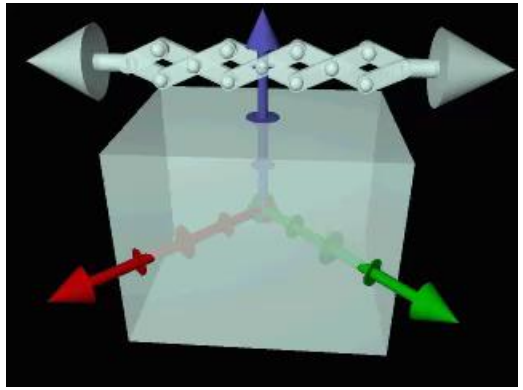
- Canonical objects
  - Local referential
    - Depending on the shape of the object
    - Unit parameters (radius, height…)
  - No intrinsic modification
    - Apply global geometrical transformations

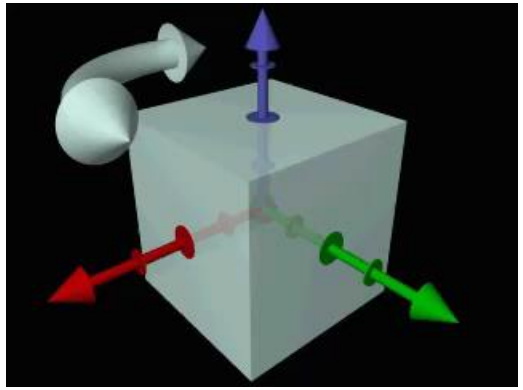


Scene modeling

- Simple geometrical transformations
  - translation : displacement of objects
  - scale : change the size along axes
  - rotations : rotate object around axes







Simple geometrical transformations

- Translation
  - add a vector $\vec{u}$ to all points of the object

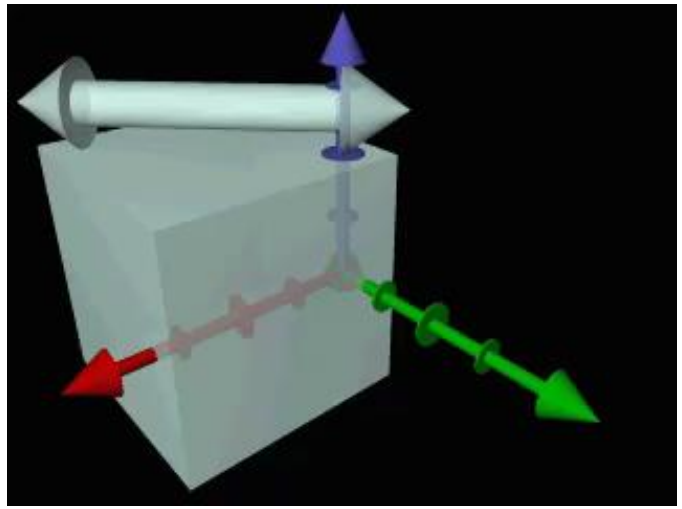
$$t_{\vec{u}}(P) = P + \vec{u}$$

➢ motion of points

  - But no effect if applied to a vector

$$t_{\vec{u}}(\vec{v}) = \vec{v}$$

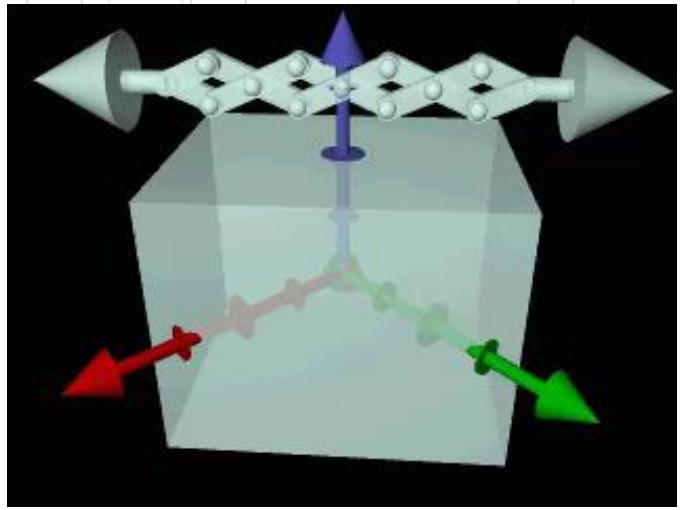
➢ dimensions and angles unchanged



Simple geometrical transformations

- Scale
  - multiply point or vector components by coefficients

$$h_{(a,b)}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax \\ by \end{pmatrix}$$

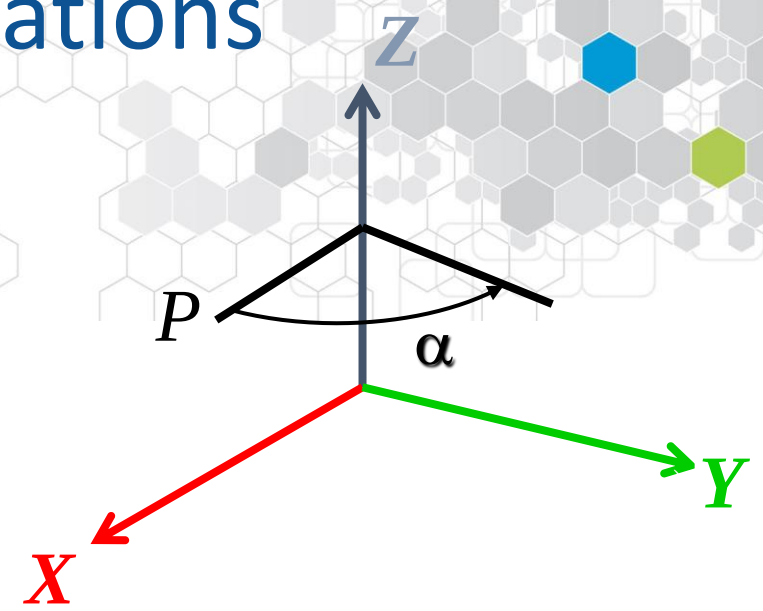


- Move points relatively to the origin,
- Scale the lengths,
- Change angles if a!=b
- Do not affect the origin

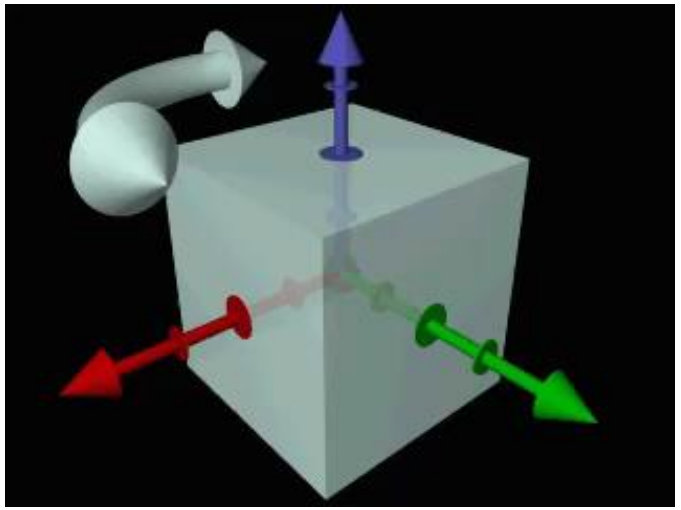
Simple geometrical transformations

- Rotation
  - rotation around an axis of a given angle.

➢ Do not affect axis points

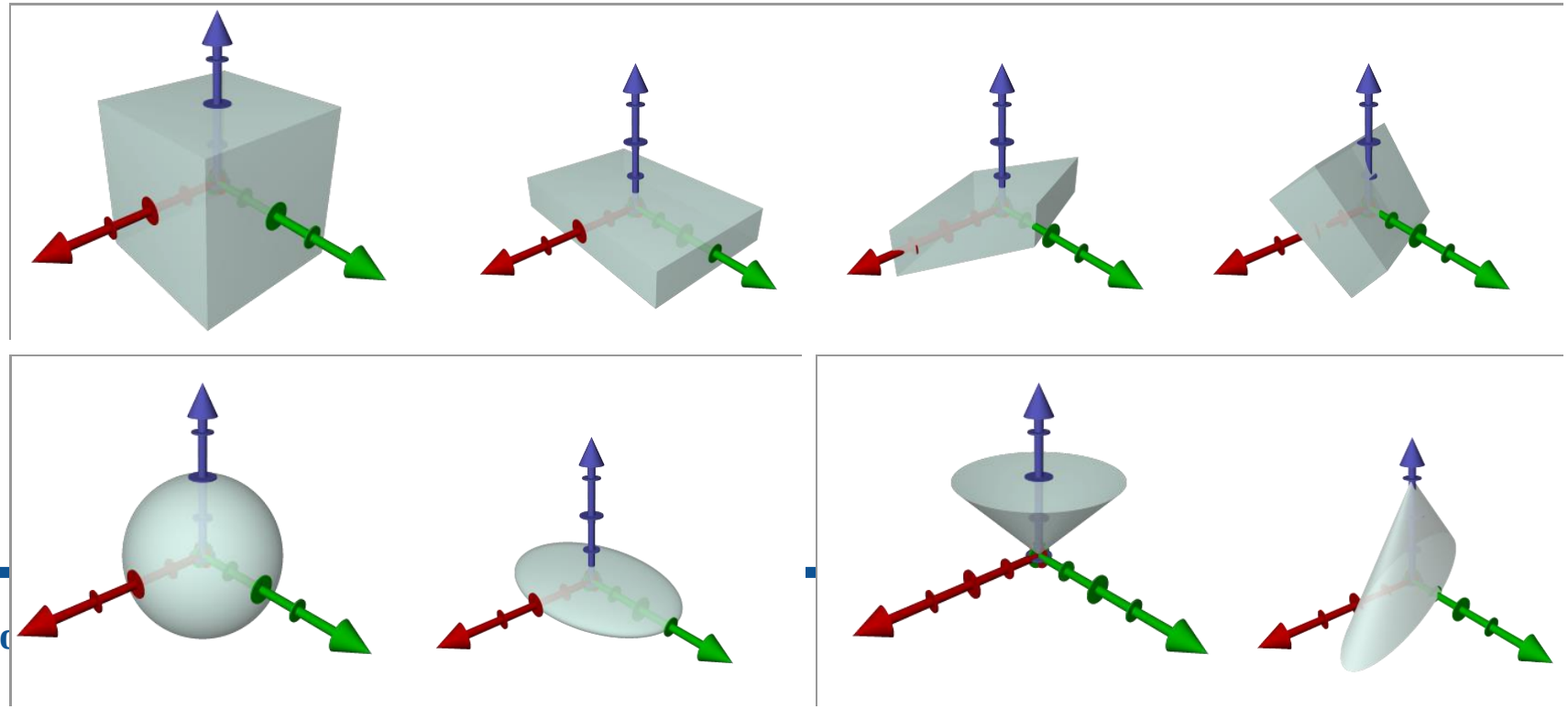


$$r_{Z(\alpha)}\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} \cos(\alpha)x - \sin(\alpha)y \\ \sin(\alpha)x + \cos(\alpha)y \\ z \end{pmatrix}$$



Canonical object transformations

- We get a family of objects
  - sphere -> ellipsoids
  - cube -> parallelepipeds
  - cylinder -> elliptic base cylinders



Geometrical transformations

- Transformation in R^3

$$r_{Z(\alpha)}(P) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} P$$

$$h_{(a,b,c)}(P) = \begin{pmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{pmatrix} P$$

$$t_{\vec{u}}(P) = P + \vec{u}$$

Common expression: using homogeneous matrices

Homogeneous matrices

- Problem: write a matrix that applies a linear combination in R^3

$$M \times P \Leftrightarrow A \times P + B$$

- Solution: using an homogenous expression of dimension 4

$$\begin{pmatrix} \begin{pmatrix} & A & \\ & & \\ 0 & 0 & 0 \end{pmatrix} & \begin{pmatrix} B \\ \\ 1 \end{pmatrix} \end{pmatrix} \times \begin{pmatrix} P \\ \\ 1 \end{pmatrix} = A \times P + B$$

Homogeneous matrices

- Translation homogeneous matrices
  - Applied to a point P

$$T_{\vec{u}}P = \begin{pmatrix} 1 & 0 & 0 & \vec{u}_x \\ 0 & 1 & 0 & \vec{u}_y \\ 0 & 0 & 1 & \vec{u}_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} P \\ 1 \end{pmatrix} = P + \vec{u}$$

  - Applied to a vector V

$$T_{\vec{u}}\vec{v} = \begin{pmatrix} 1 & 0 & 0 & \vec{u}_x \\ 0 & 1 & 0 & \vec{u}_y \\ 0 & 0 & 1 & \vec{u}_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} \vec{v} \\ 0 \end{pmatrix} = \vec{v}$$

- The expression of a **point** and a **vector** are different using homogeneous coordinates!

Homogeneous matrices

- Rotation around Z axis homogeneous matrix

$$R_{Z(\alpha)} = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Scale homogeneous matrix

$$H_{(a,b,c)} = \begin{pmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Composing transformations

- If we apply several transformations
  - Can be coded in a simple matrix,
  - Is the product of transformation matrices,
  - Be careful with the order of transformations!

Rotation of angle α around Z followed by a translation of vector $\vec{u}$

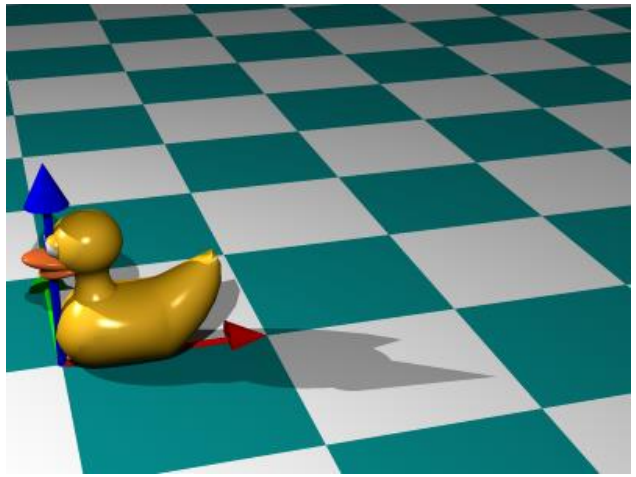
$$t_{\vec{u}}\left(r_{Z(\alpha)}\left(Objet\right)\right) \Leftrightarrow T_{\vec{u}}\left(R_{z(\alpha)}P_{objet}\right) = \left(T_{\vec{u}} \times R_{z(\alpha)}\right)P_{objet}$$

Be careful with the order

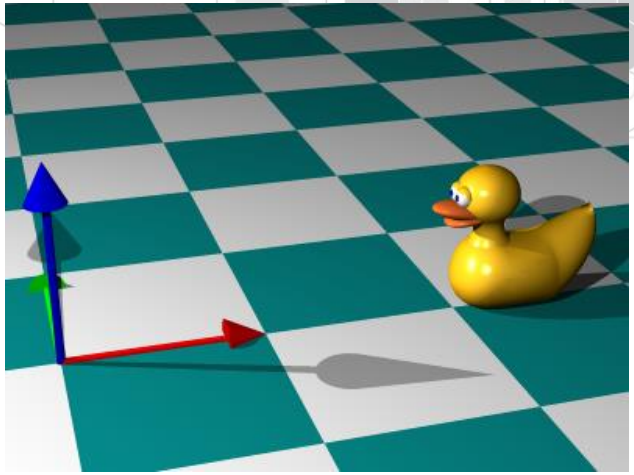
$$\left(T_{\vec{u}} \times R_{z(\alpha)}\right)P_{objet} \neq \left(R_{z(\alpha)} \times T_{\vec{u}}\right)P_{objet}$$

Examples of compositing

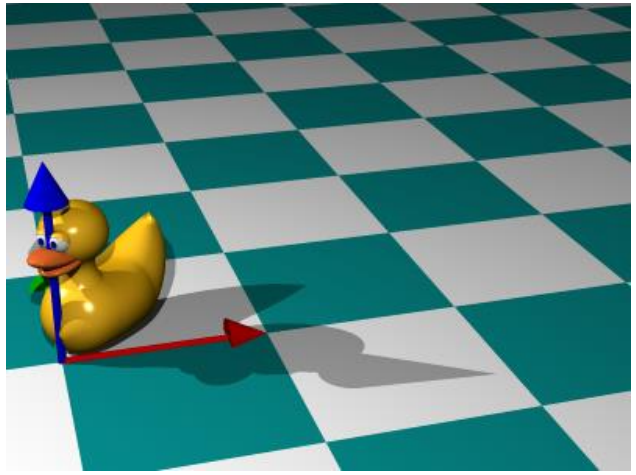
Initial objet



$T_{2\vec{X}} P_{duck}$



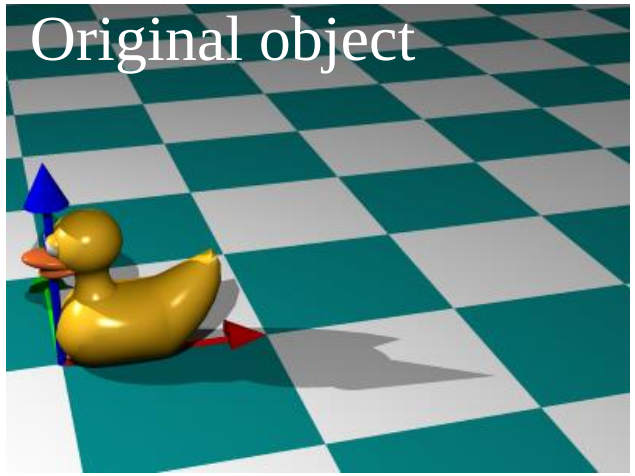
$R_{z(45°)} P_{duck}$

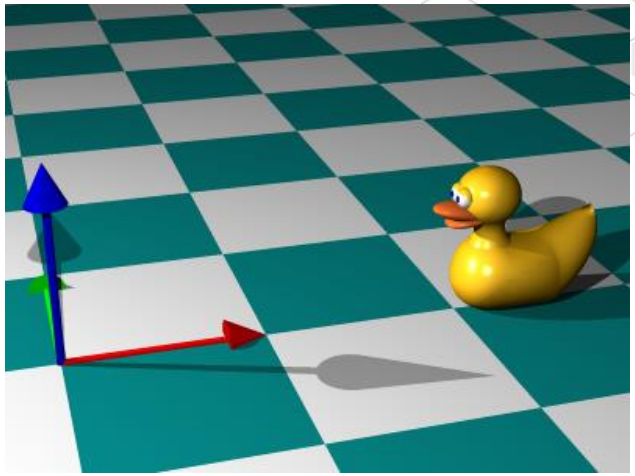


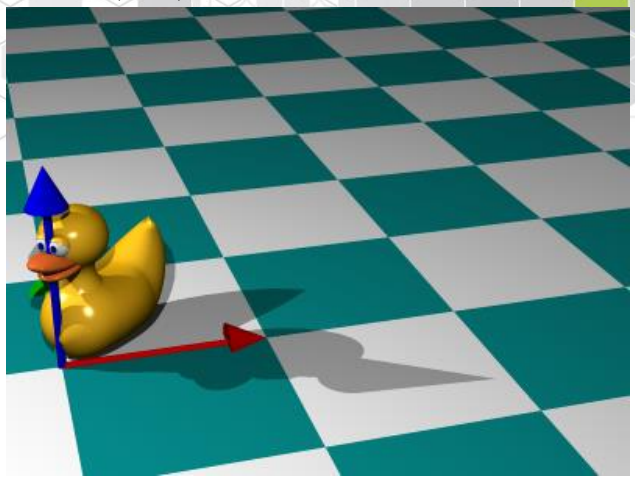
First order

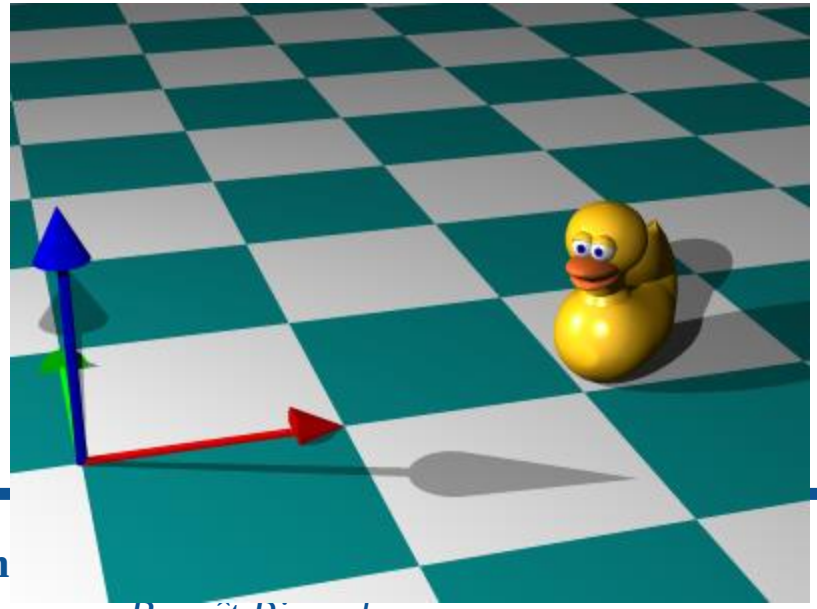
$T_{2\vec{X}} P_{duck}$

$R_{z(45°)} P_{duck}$







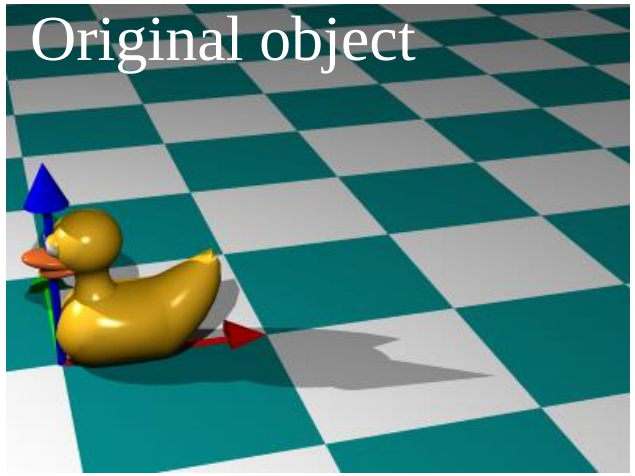


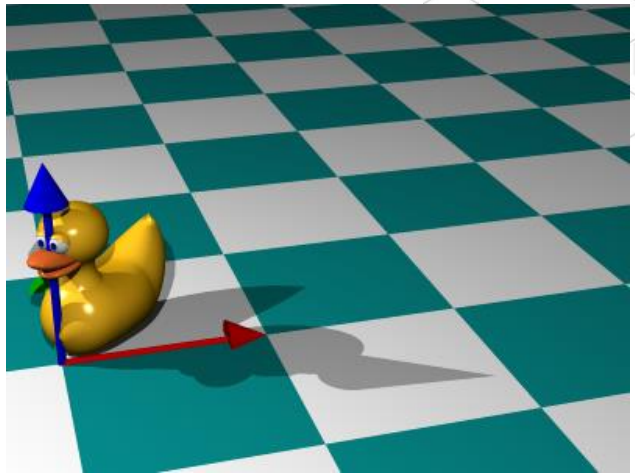
$\left(T_{2\vec{X}} \times R_{z(45°)}\right) P_{duck}$

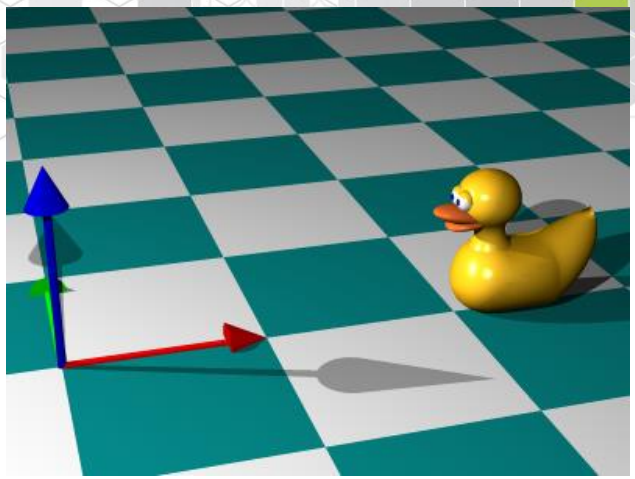
Second order

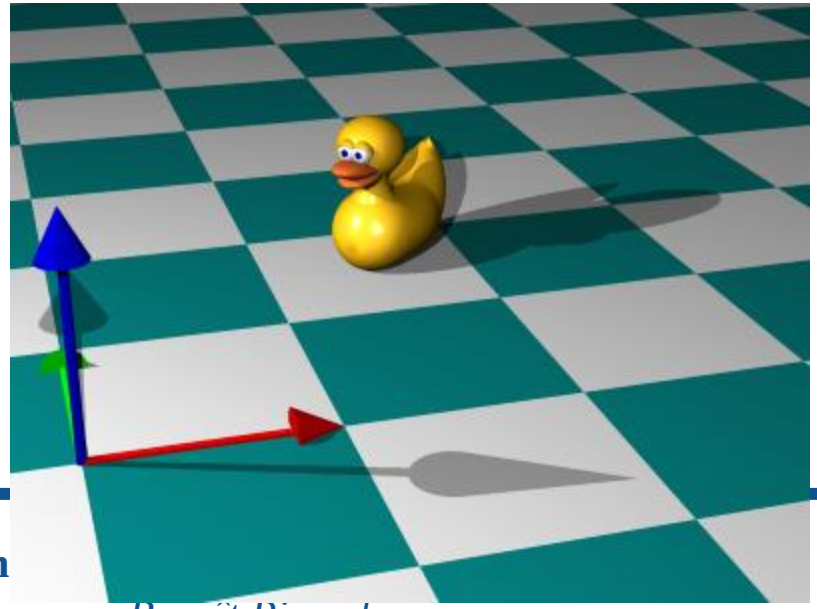
$R_{z(45°)}P_{duck}$

$T_{2\vec{X}}P_{duck}$





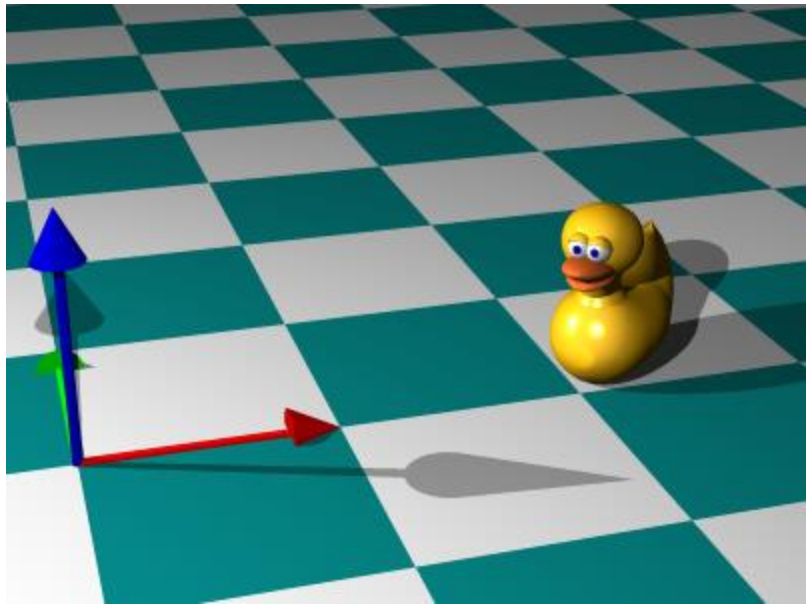




$\left(R_{z(45°)} \times T_{2\vec{X}}\right)P_{duck}$

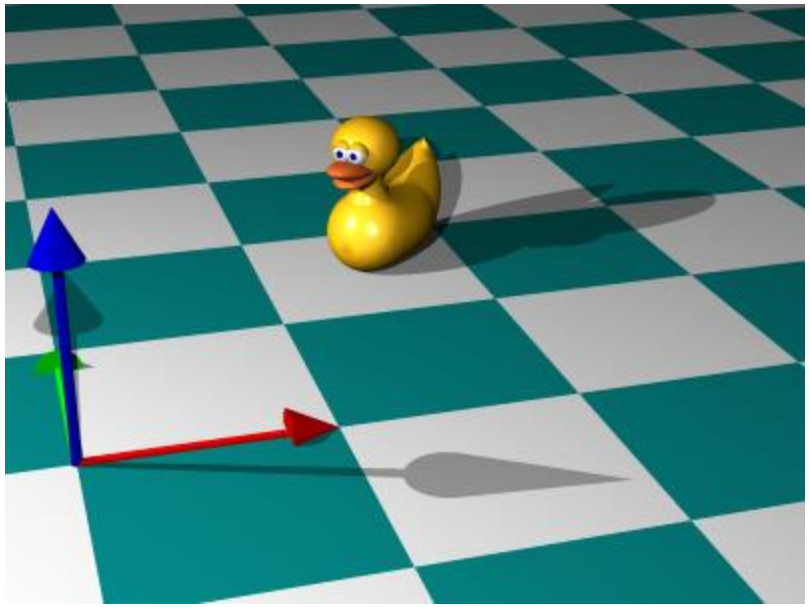
Order of combination

$$\left(T_{2\vec{X}} \times R_{z(45°)}\right)P_{duck}$$



$\neq$

$$\left(R_{z(45°)} \times T_{2\vec{X}}\right)P_{duck}$$



OpenGL scene modelisation

- Uses Homogeneous matrices
  - GL_PROJECTION_MATRIX: for the camera and projection into the scree,
  - GL_MODELVIEW_MATRIX: for the scene
  - glInitMatrix(): set the current matrix to Identity
  - glTranslatef(x,y,z): multiply the current matrix with the translation matrix
  - glRotatef(angle,x,y,z): multiply the current matrix with the rotation matrix
  - glScalef(x,y,z): multiply the current matrix with the scale matrix
  - glPushMatrix(): store the current matrix into a stack
  - glPopMatrix(): set the matrix at the top at the stack to the current matrix and pop it.

Example of usage

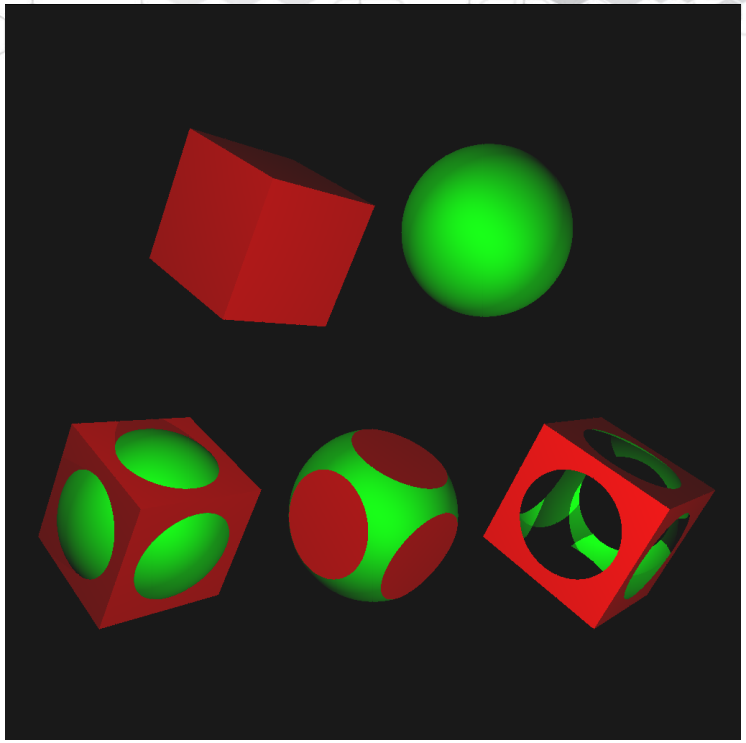
```
glPushMatrix();
glMaterialfv(GL_FRONT,GL_AMBIENT_AND_DIFFUSE,blue);
glutSolidCylinder(0.02,2.0,20,5);
glPushMatrix();
glRotatef(-90.0,1,0,0);
glMaterialfv(GL_FRONT,GL_AMBIENT_AND_DIFFUSE,green);
glutSolidCylinder(0.02,2.0,20,5);
glPopMatrix();
glRotatef(90.0,0,1,0);
glMaterialfv(GL_FRONT,GL_AMBIENT_AND_DIFFUSE,red);
glutSolidCylinder(0.02,2.0,20,5);
glPopMatrix();
```

Scene modeling

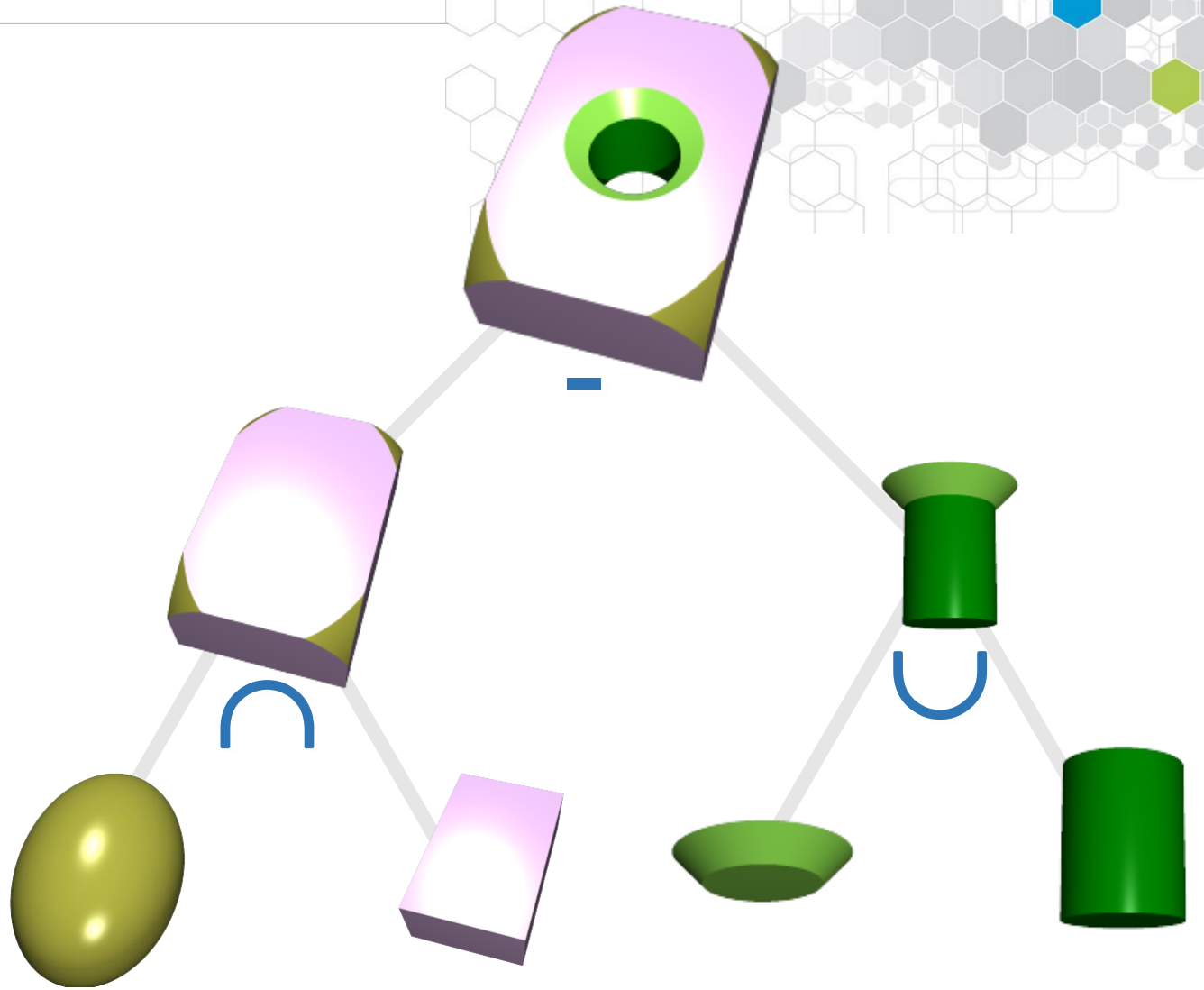
- CSG Model : Constructive Solid Geometry Tree
  - Describe a scene by a tree
- CSG Tree
  - Leaves:
    - canonical objects
  - Internal nodes:
    - Sub-trees compositing operators
      - Union
      - Intersection
      - Difference
    - Geometrical operators
      - Translation
      - Rotation
      - Scale

Compositing operators

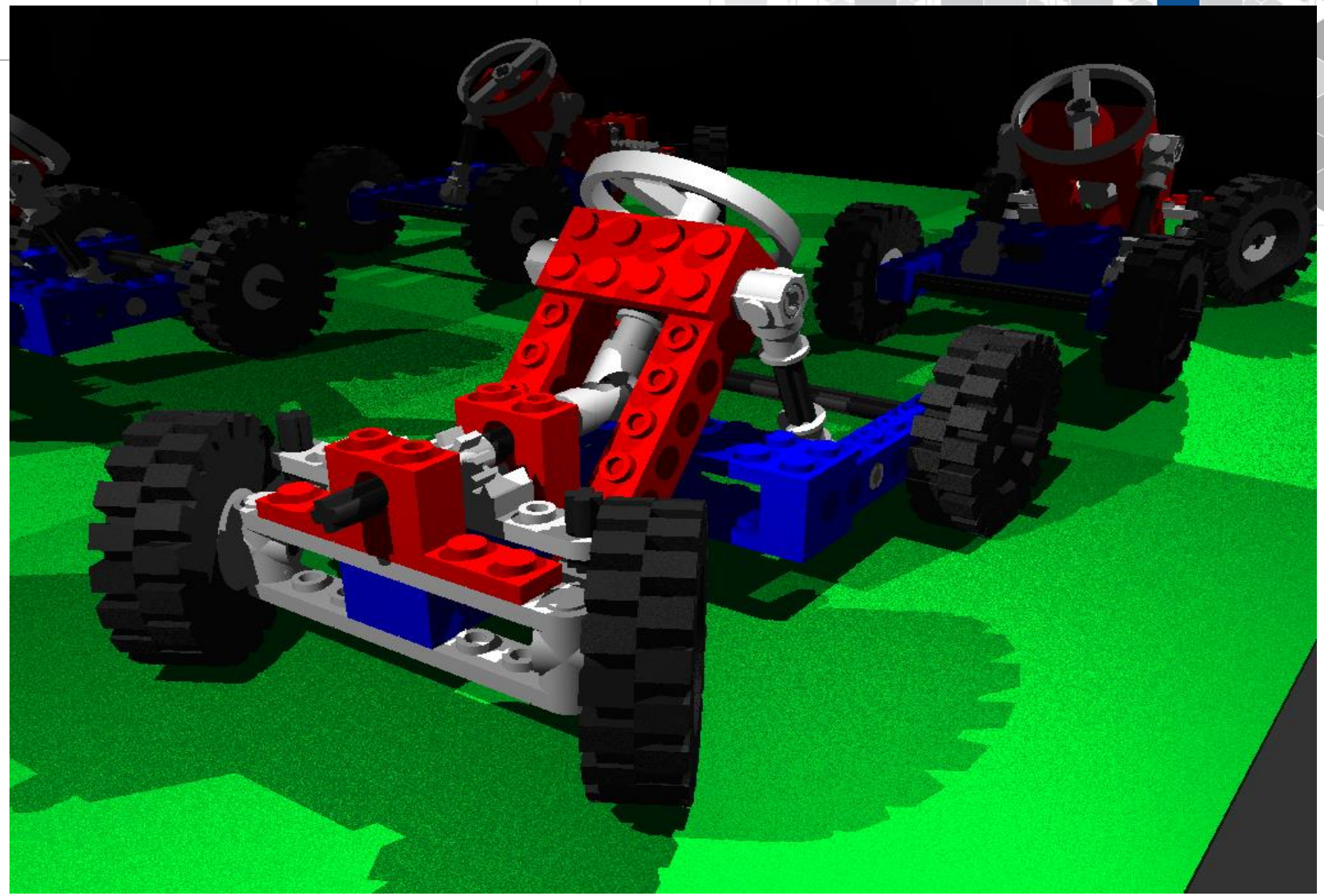
- Union
  - volume filled by at least one of the sub-trees.
- Intersection
  - Common volume of sub-trees.
- Différence
  - volume of the first sub-tree but not filled by the others.



CSG Tree



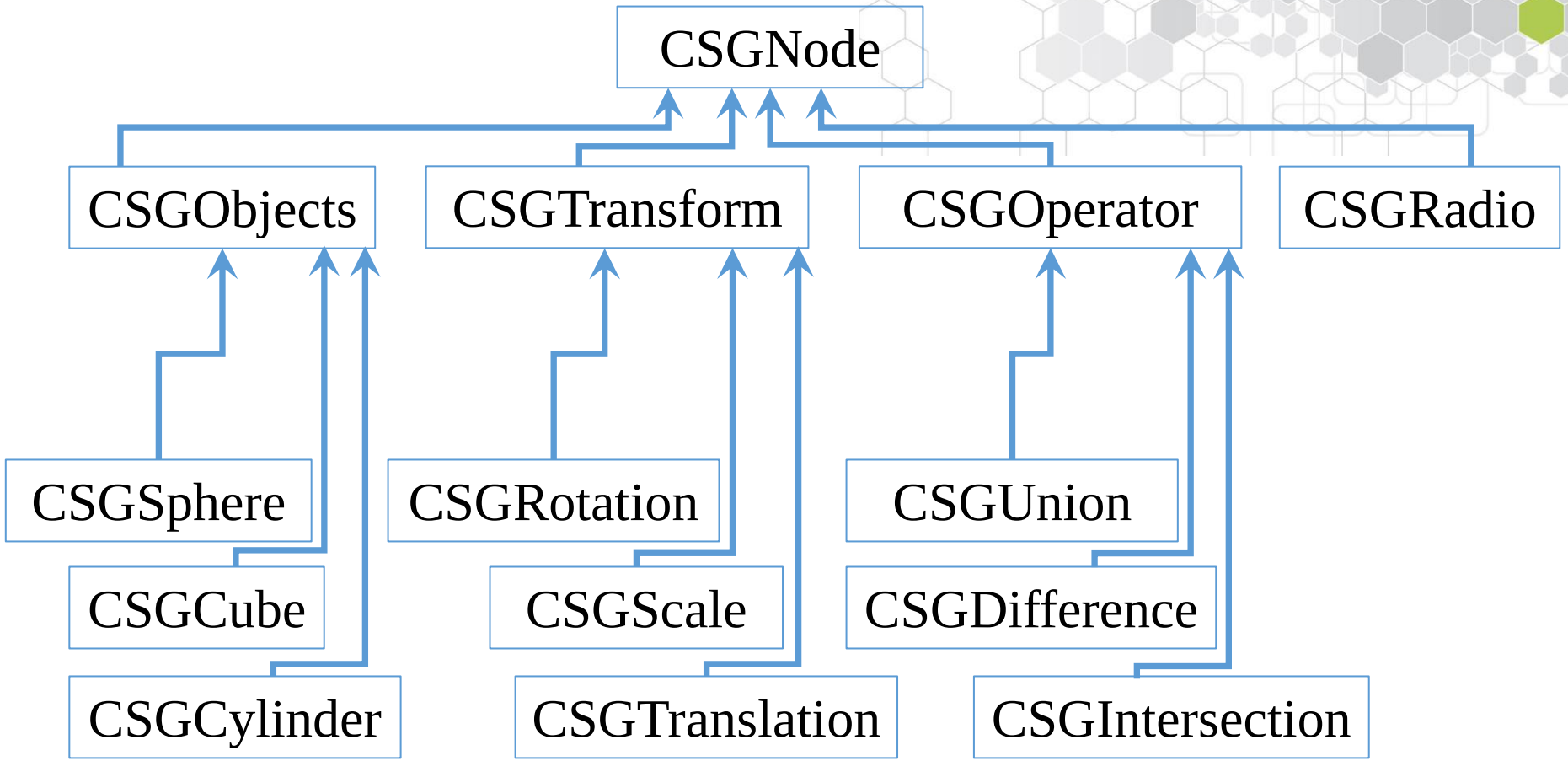
Full scene



Programming CSG trees

- Definition of a set of classes
  - Inheritance and polymorphism
    - We define a main class CSGNode
    - We define derived classes for each kind of node (CSGUnion, CSGSphere,…)
  - Attributs:
    - CSGNode : int id
    - CSGTransform : Matrix mat,mat_1
    - CSGObject : string nom
  - Methods
    - glDraw() to draw each node (internal node too!)
    - Picking() to calculate the intersections beween a ray and the scene

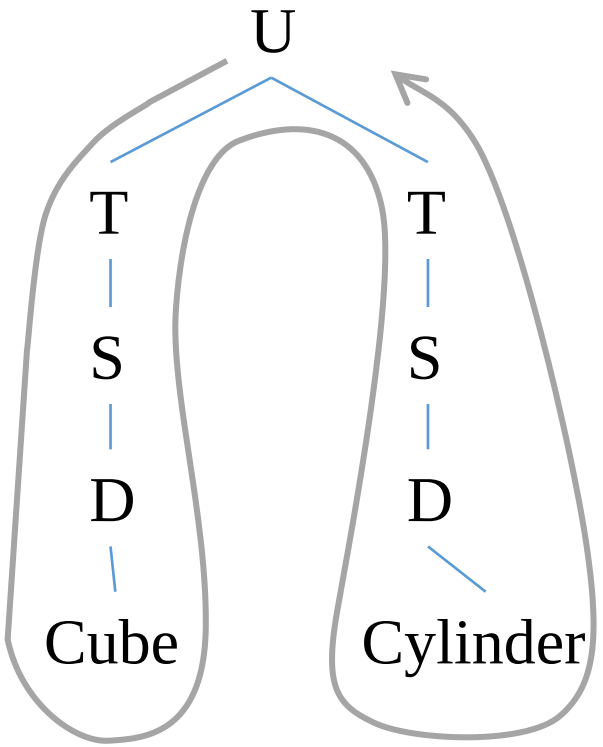
Classes


CSGNode
CSGObjects
CSGTransform
CSGOperator
CSGRadio
CSGSphere
CSGCube
CSGCylinder
CSGRotation
CSGScale
CSGTranslation
CSGUnion
CSGDifference
CSGIntersection

CSG tree / associated word

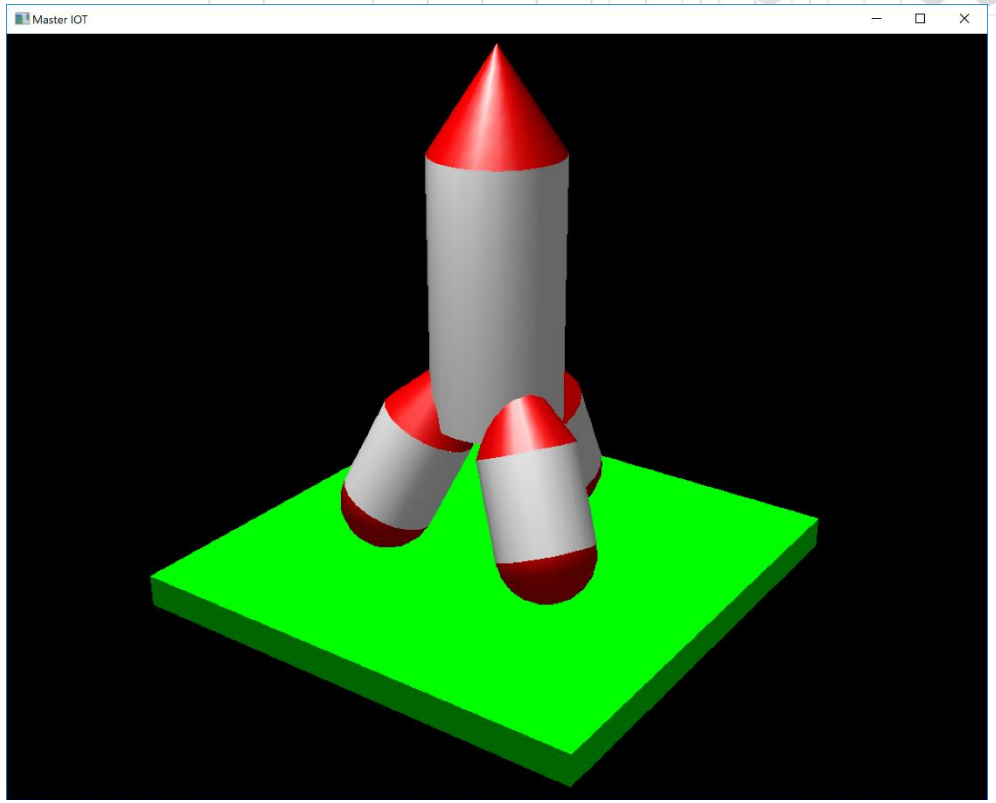
- We can write an instance of a CSG tree by a word
- Prefix depth first search algorithm
  - keyword + parameters

U 2	*union + 2 children*
T -1,0,-2.1	*translation + vecteur*
S 5,2,0.1	*scale + coefficients*
A&D 1.0,0.0,0.5	*diffusion + color*
CUB sol	*Cube + name*
T 1,0,-2	*translation + vector*
S 0.5,0.5,4	*scale + coefficients*
A&D 0.0,1.0,0.5	*diffusion + color*
CYL tube	*Cylinder + name*



CSG tree / associated word

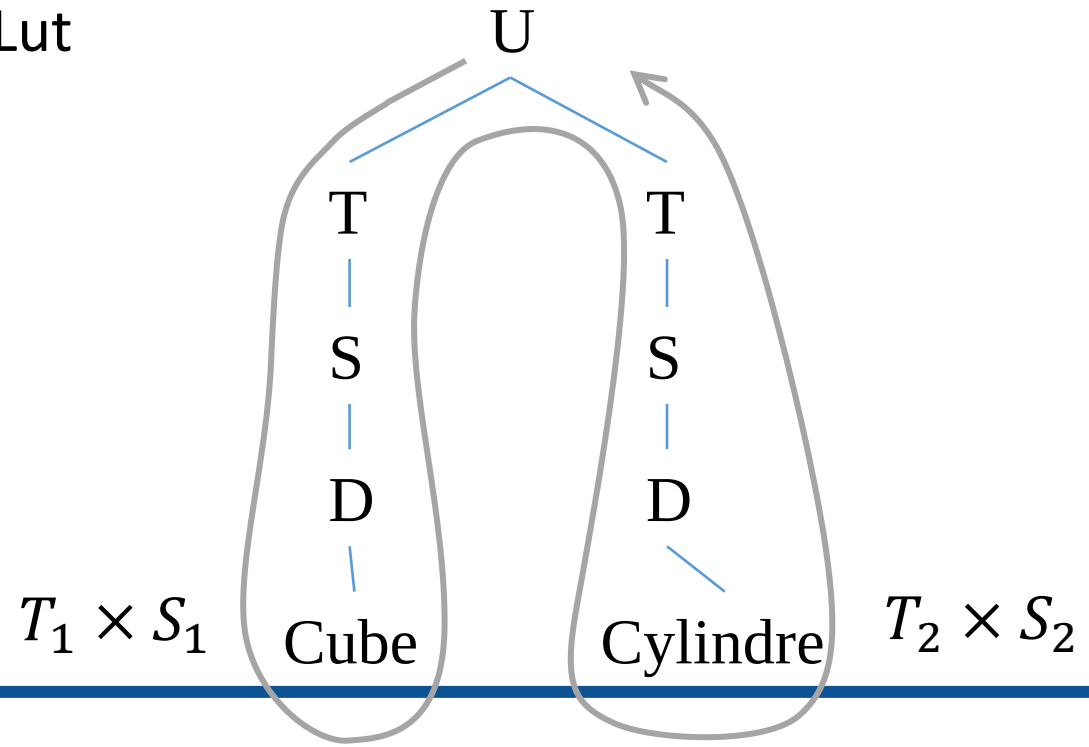
```
U 2
A&D 0.0,1.0,0.0
SPC 1.0,1.0,1.0 100.0
T 0,0,-4.93
S 15,15,1
CUB sol
A&D 1.0,1.0,1.0
SPC 1.0,1.0,1.0 20.0
U 5
S 2,2,8
A&D 1.0,1.0,1.0
CYL corps
T 0,0,8
S 2,2,3
A&D 0.8,0.0,0.0
CON capsule
T 2.034,0,-0.25
RY -26.565
U 3
S 1.5,1.5,3
A&D 0.8,0.0,0.0
CON pied01
T 0,0,-3
S 1.5,1.5,3
A&D 1.0,1.0,1.0
CYL tubePied01
T 0,0,-3
S 1.5,1.5,1.5
A&D 0.8,0.0,0.0
SPH basePied01
RZ 120
T 2.034,0,-0.25
RY -26.565
U 3
S 1.5,1.5,3
A&D 0.8,0.0,0.0
CON pied02
T 0,0,-3
S 1.5,1.5,3
A&D 1.0,1.0,1.0
CYL tubePied02
T 0,0,-3
S 1.5,1.5,1.5
A&D 0.8,0.0,0.0
SPH basePied02
RZ 240
T 2.034,0,-0.25
RY -26.565
U 3
S 1.5,1.5,3
A&D 0.8,0.0,0.0
CON pied03
T 0,0,-3
S 1.5,1.5,3
A&D 1.0,1.0,1.0
CYL tubePied03
T 0,0,-3
S 1.5,1.5,1.5
A&D 0.8,0.0,0.0
SPH basePied03
```



glDraw Method

- Problème :
  - Transmettre les transformations géométriques pour dessiner les objets
  - Construction du produit matriciel de transformations géométriques
  - Utilisation des outils OpenGL/GLut

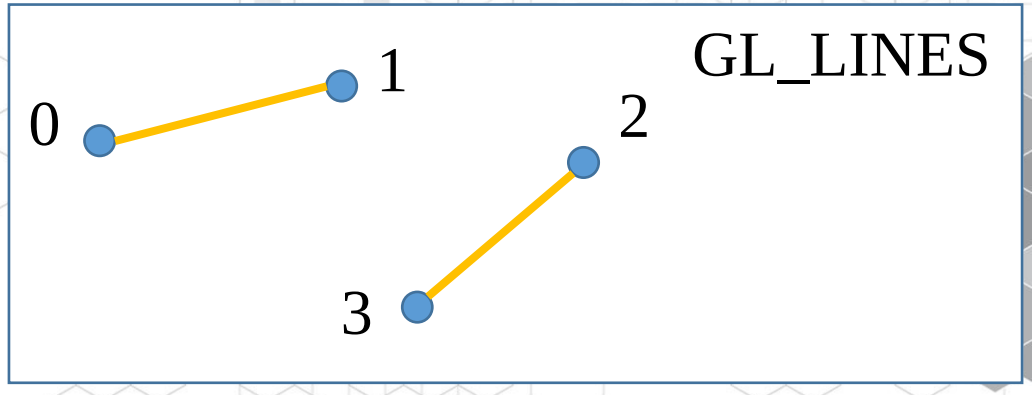
```
glPushMatrix();
gltranslatef(-1,0,-2.1);
glScalef(5,1,0.1);
glutSolidCube(1);
glPopMatrix();

glPushMatrix();
gltranslatef(1,0,-2);
glScalef(0.5,0.5,4);
glutSolidCylinder(1,1,…);
glPopMatrix();
```

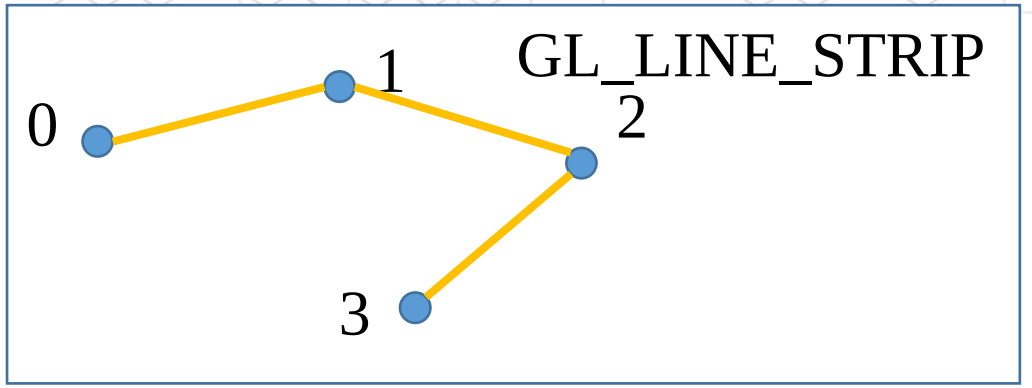


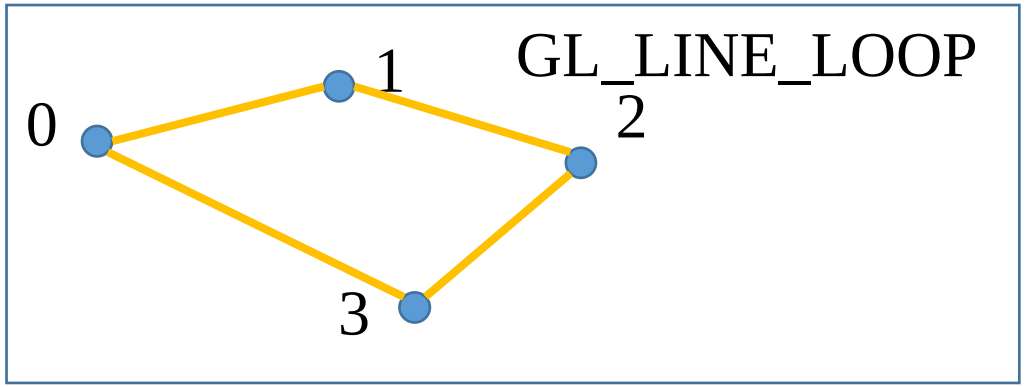
Basic 2D functions

- OpenGL drawing tools
  - Line width
  - Basic 2D shapes:
    - GL_LINES (2N vertices),
    - GL_LINE_STRIP (N vertices)
    - GL_LINE_LOOP (N vertices),
    - GL_TRIANGLE (3N vertices),
    - GL_QUADS (4N verticies),
    - GL_POLYGON (N vertices)

```
glLineWidth(3);
```







Geometrical transformations

- Translation $T(x, y)$ of the futur drawing

```
glTranslatef(cx,cy,0.0f);
```

- Rotation $R(alpha)$ of the futur drawing (alpha in degree)

```
glRotatef(alpha,0.0f,0.0f,1.0f);
```

- Scaling $S(sx, sy)$ of the futur drawing

```
glScalef(sx,sy,1.0f);
```

- Rotate then translate:

```
glPushMatrix();
glTranslatef(cx,cy,0.0f);
glRotatef(alpha,0.0f,0.0f,
Drawing
glPopMatrix();
```

Drawing a circle?

```
glPushMatrix();
glTranslatef(cx,cy,0.0f);
glBegin(GL_LINE_LOOP);
    for (int j=0; j<Npts; j++) {
        double a = 2.0*j*M_PI/Npts;

glVertex2d(pointSize*cos(a),pointSize*sin
(a));
    }
glEnd();
glPopMatrix();
```